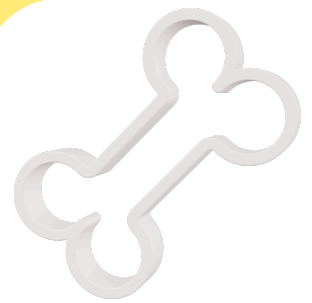
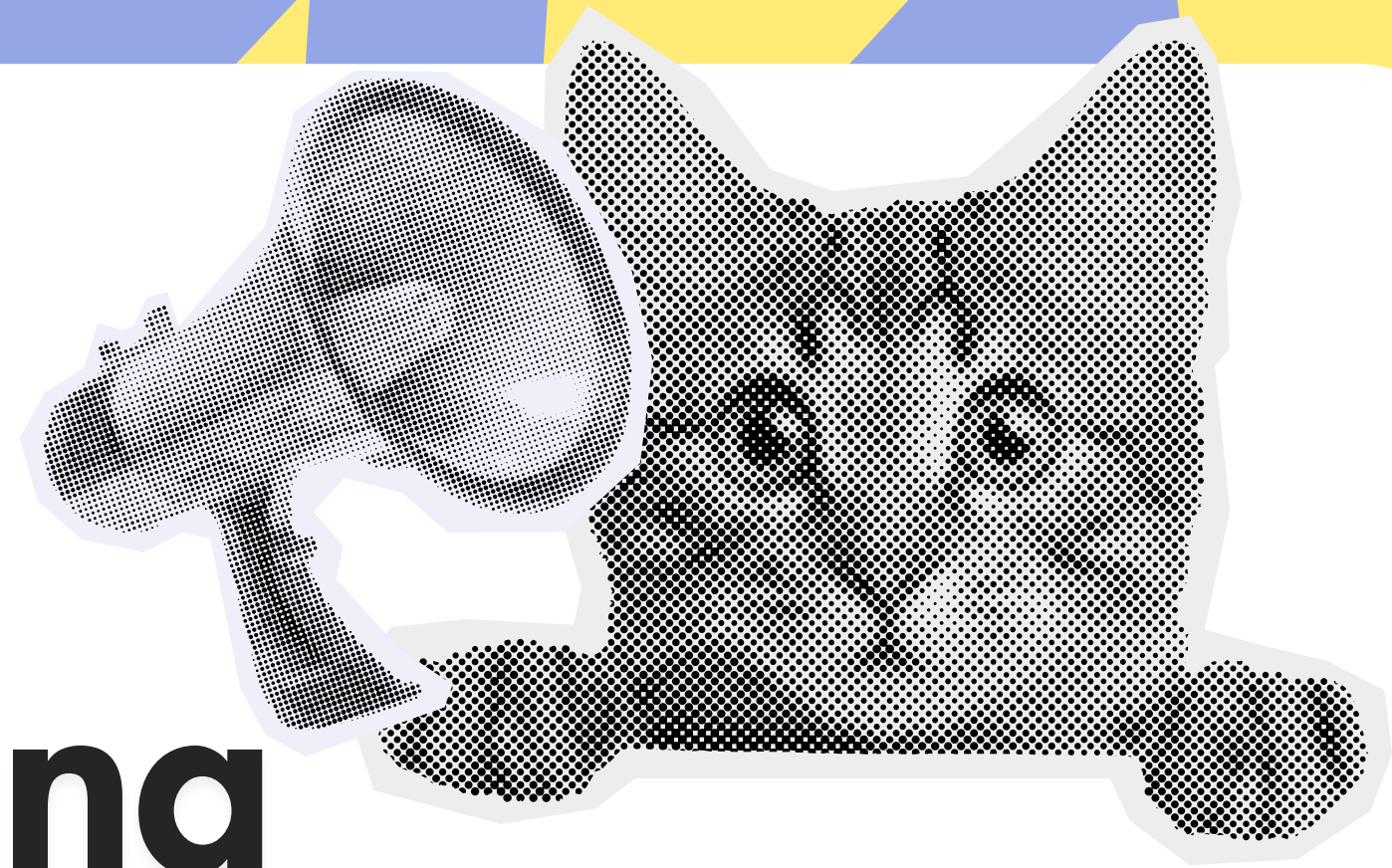




งานกลุ่ม

Image Dataset Cleaning & Preprocessing Pipeline

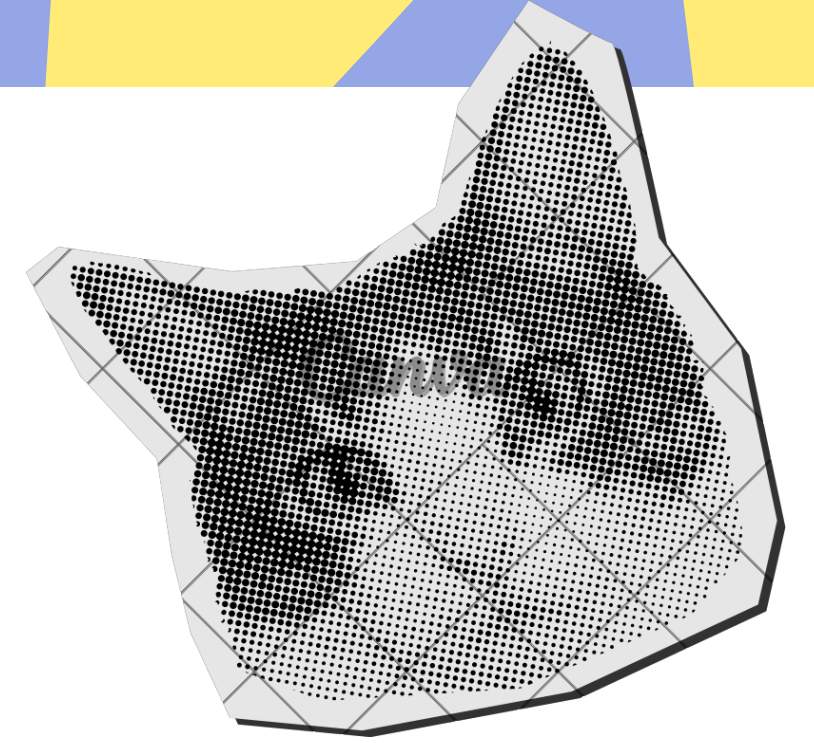


หัวข้อ Cats and Dogs image classification





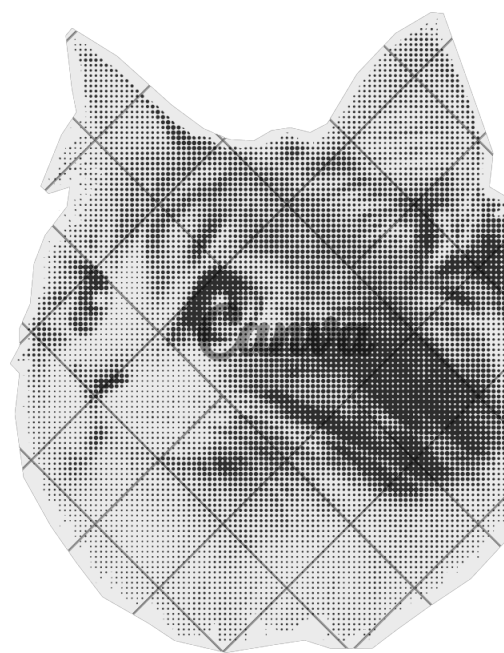
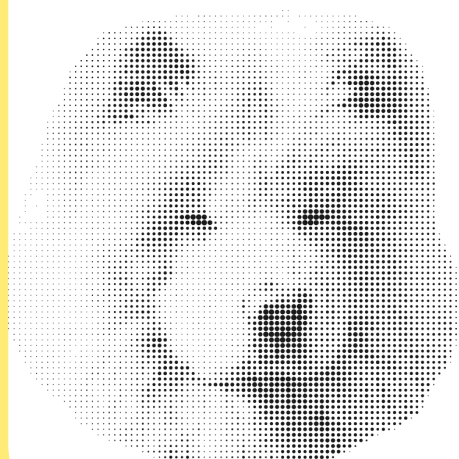
รายชื่อกลุ่ม



นาย ธนวัฒน์ รุ่งผดุงพันธ์ 077 (feature/collect/split)

นาย ธนาวิช ภัคดี 078 (feature/preprocessing)

นาย นนทพันธ์ สุขกำเนิด 079 (feature/eda)



Dataset

ใช้ Dataset

samuelcortinhas/cats-and-dogs-image-classification

จาก kaggle

เป็นข้อมูลภาพแมวและสุนัขมีความหลากหลายทั้งรูปทรง สี และท่าทาง

จำนวนภาพทั้งหมด 697รูป

มี 2 Class คือ -แมว -สุนัข

ขนาดไฟล์รวม 68 MB

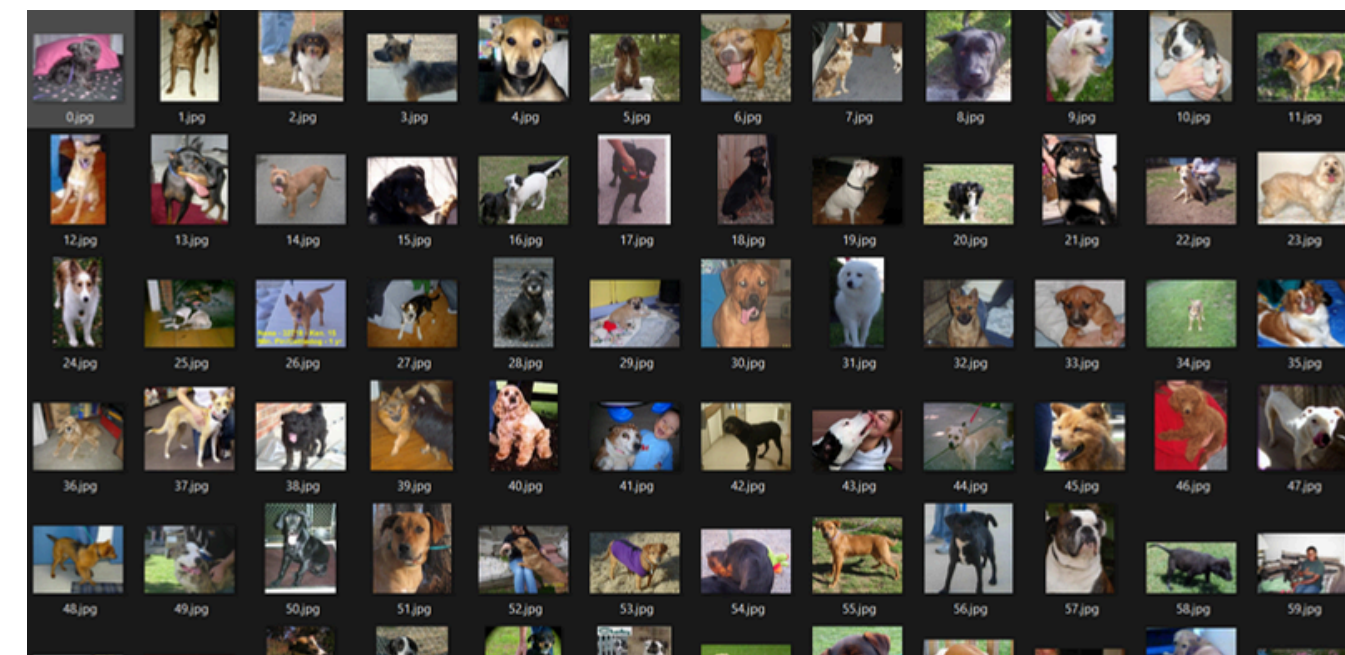
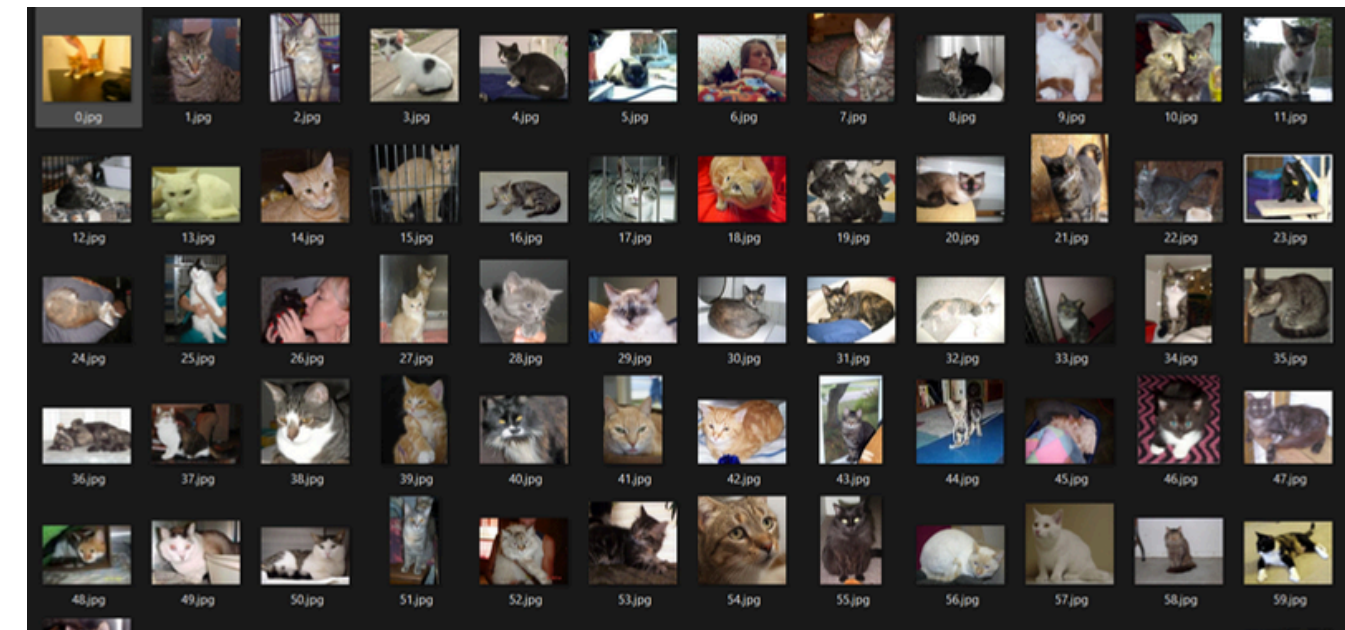


Data Collection

จะมีการเลือก Image Dataset จาก Kaggle ที่มีขนาดเหมาะสม ไม่เล็กหรือใหญ่เกินไป ทั้งหมด 697 รูป และ เขียน Python Script ที่ใช้ kaggle API เพื่อดาวน์โหลดและเก็บไว้ในโฟลเดอร์ที่ต้องการ

วิธีการทำงานของ Code

จะมีการดึงรูปภาพมาใส่ในโฟลเดอร์ที่เตรียมไว้ เพื่อให้งานส่วนอื่นๆสามารถนำรูปไปใช้ และ ทำงานได้ต่อเนื่องมากขึ้น

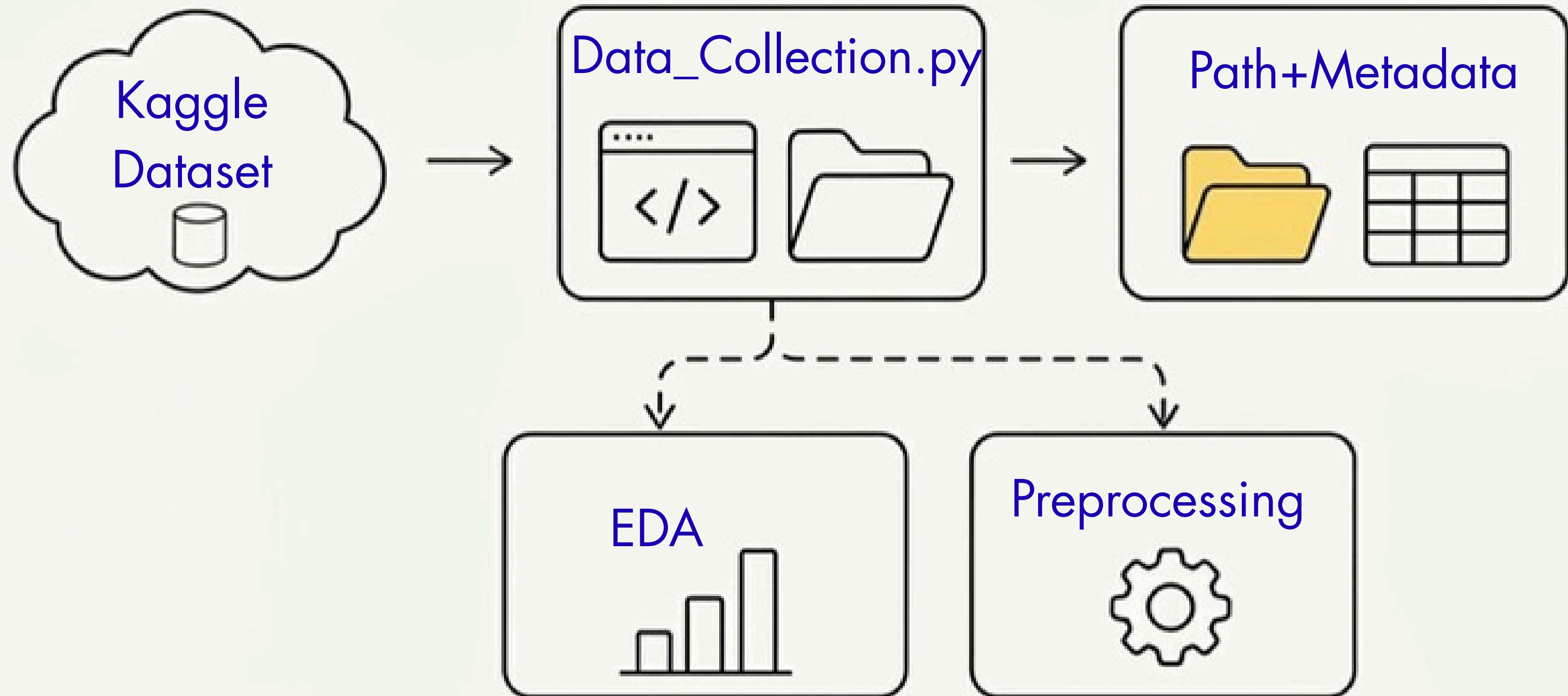


ส่วนของ.gitignore

.gitignore เป็นไฟล์ที่บอกว่าไฟล์หรือโฟลเดอร์ไหน ไม่ต้องอัปขึ้น GitHub และใส่ data/ หรือ dataset/ ลงใน .gitignore เพื่อป้องกัน Dataset ถูก Commit ทำให้ GitHub เหลือเฉพาะโค้ดและไฟล์ที่จำเป็น ช่วยประหยัดพื้นที่และเพิ่มความปลอดภัย

```
1  # --- Dataset & Raw Data ---
2  Dataset-cat-and-dog-image/
3  data/
4  dataset/
5  *.zip
6  *.tar
7  *.gz
8
9  # --- Processed Outputs & Reports Data ---
10 figures/*
11 figures/.gitkeep
12
13 # --- IDE & System Files ---
14 .vscode/
15 .idea/
16 __pycache__/*
17 *.pyc
18 *.pyo
19 *.pyd
20 .DS_Store
21 Thumbs.db
22
23 # --- Secrets & API Keys ---
24 kaggle.json
25 .env
```

ภาพรวมข้อมูล

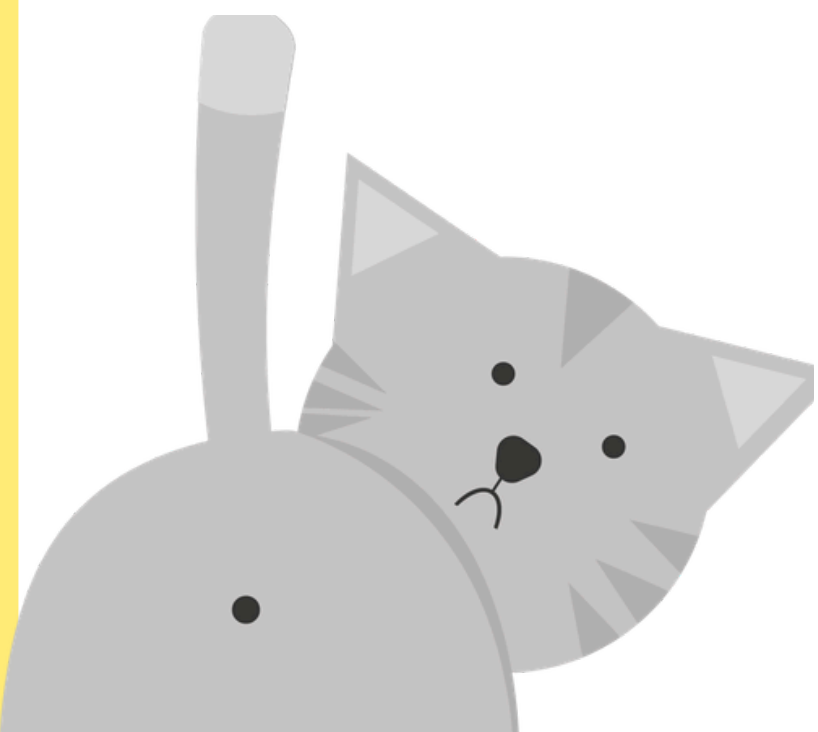




Exploratory Data Analysis (EDA)



เชิงปริมาณ & เชิงคุณภาพ



วัตถุประสงค์ที่ทำ EDA

ทำความเข้าใจ Dataset

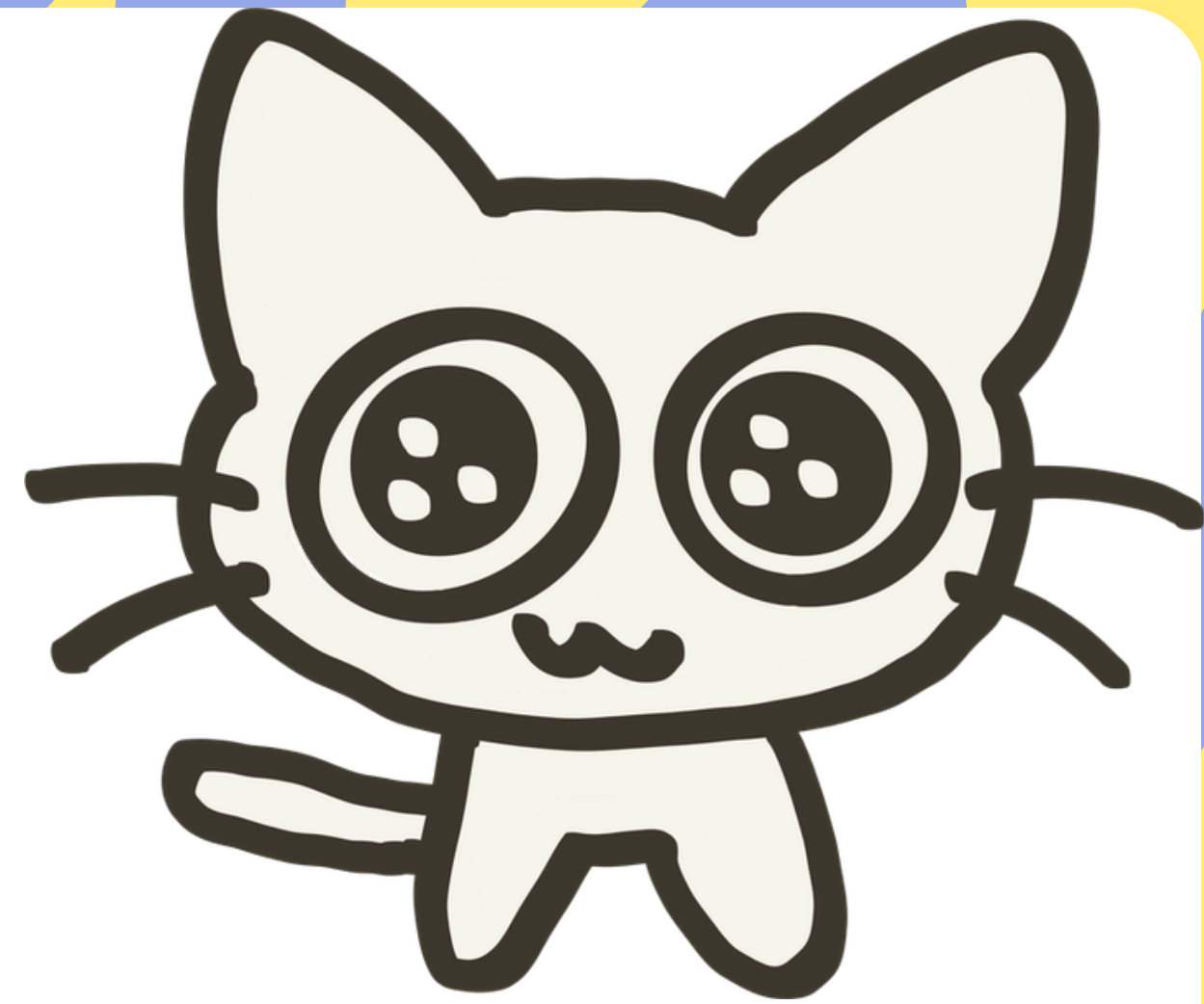
- รู้ว่ามีข้อมูลอะไรบ้าง
- มีรูปแบบและสัณฐานจำนวนเท่าไร
- รูปภาพมีขนาดและลักษณะแตกต่างกันอย่างไร

ตรวจสอบคุณภาพของข้อมูล

- ตรวจสอบว่ามีรูปที่เปิดไม่ได้หรือไฟล์เสียหรือไม่
- ตรวจสอบภาพที่ผิดปกติ
- ตรวจสอบภาพที่เบลอ มืด หรือสว่างเกินไป

ตรวจสอบความสมดุลของข้อมูล

- ดูว่าจำนวน cats และ dogs ใกล้เคียงกันหรือไม่
- ถ้าจำนวนต่างกันมาก อาจทำให้โมเดลเรียนรู้ไปทาง Class ที่มีข้อมูลเยอะกว่า



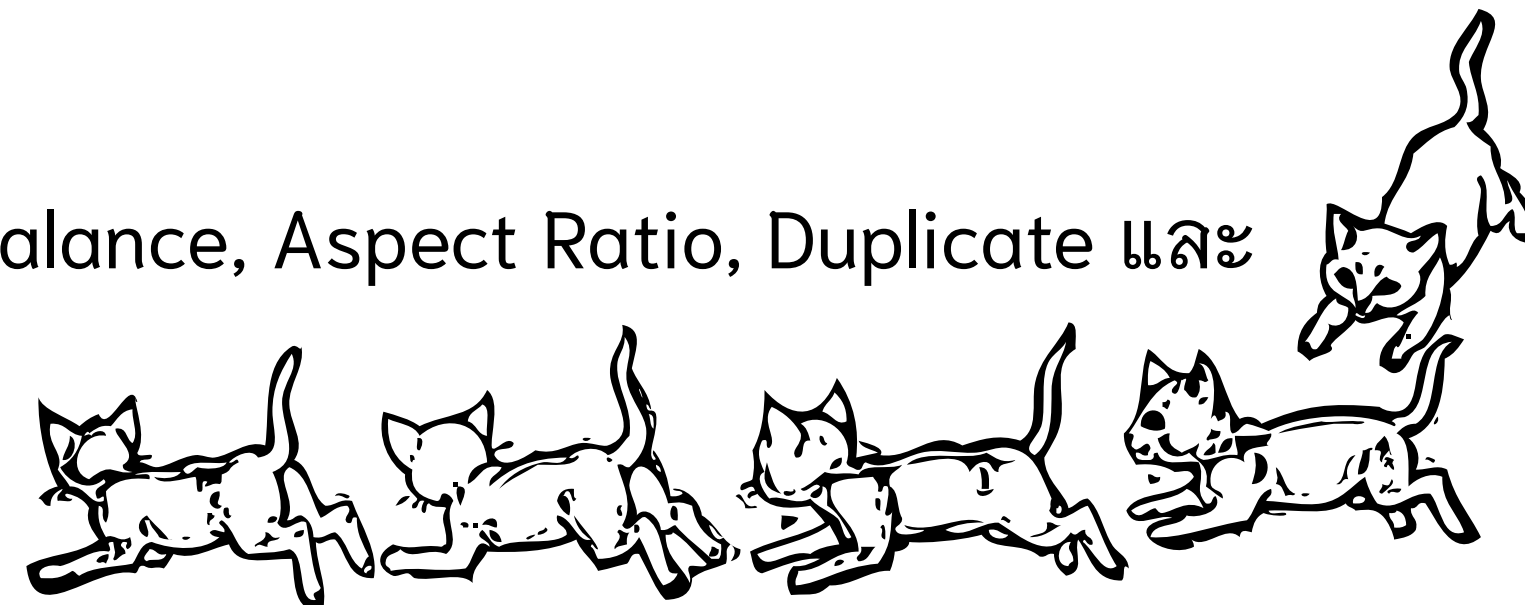
ค้นหาข้อมูลที่ผิดปกติ (Outlier)

- ภาพที่มีขนาดใหญ่มาก
- ไฟล์ที่มีขนาดใหญ่ผิดปกติ
- ภาพที่มี Aspect Ratio แปลก
- ภาพที่เบลอมาก
- ภาพที่มืดหรือสว่างมาก

นำข้อมูลไปวางแผน Data Preprocessing

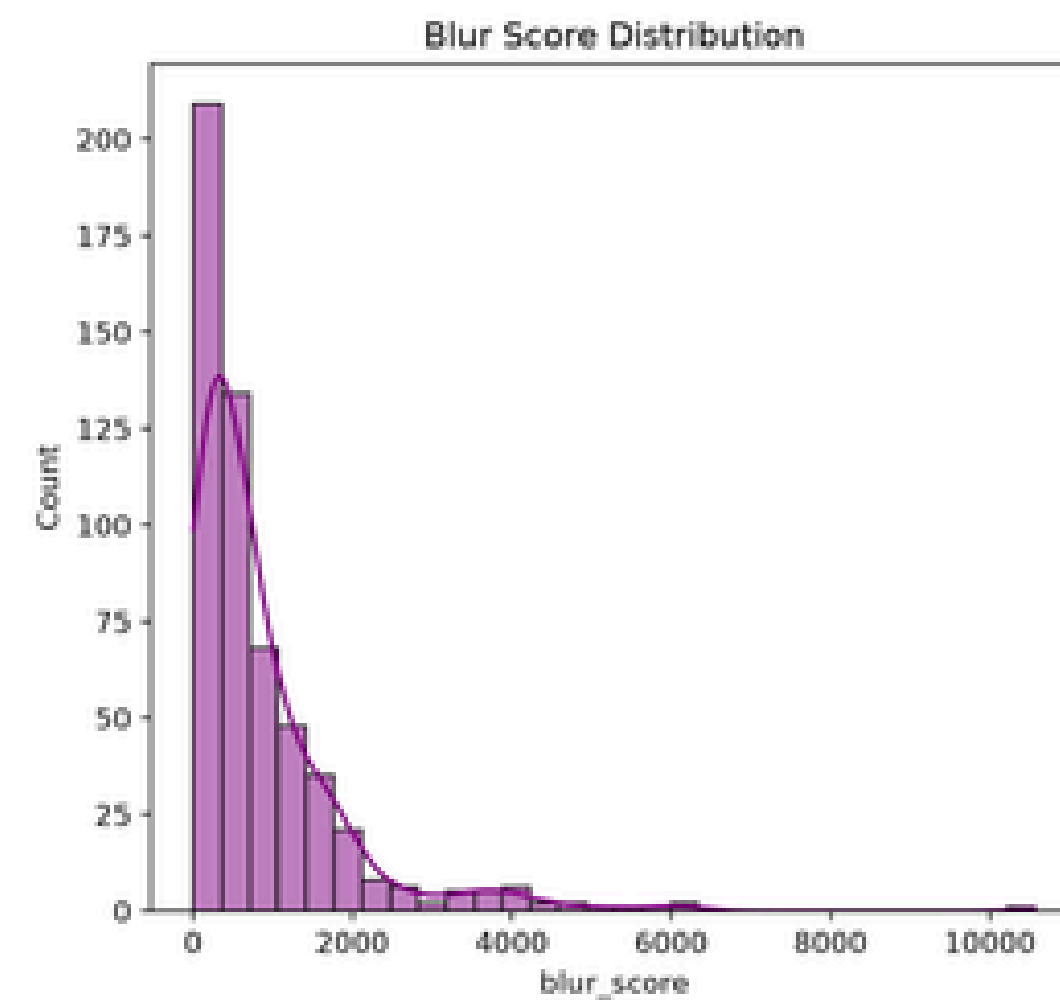
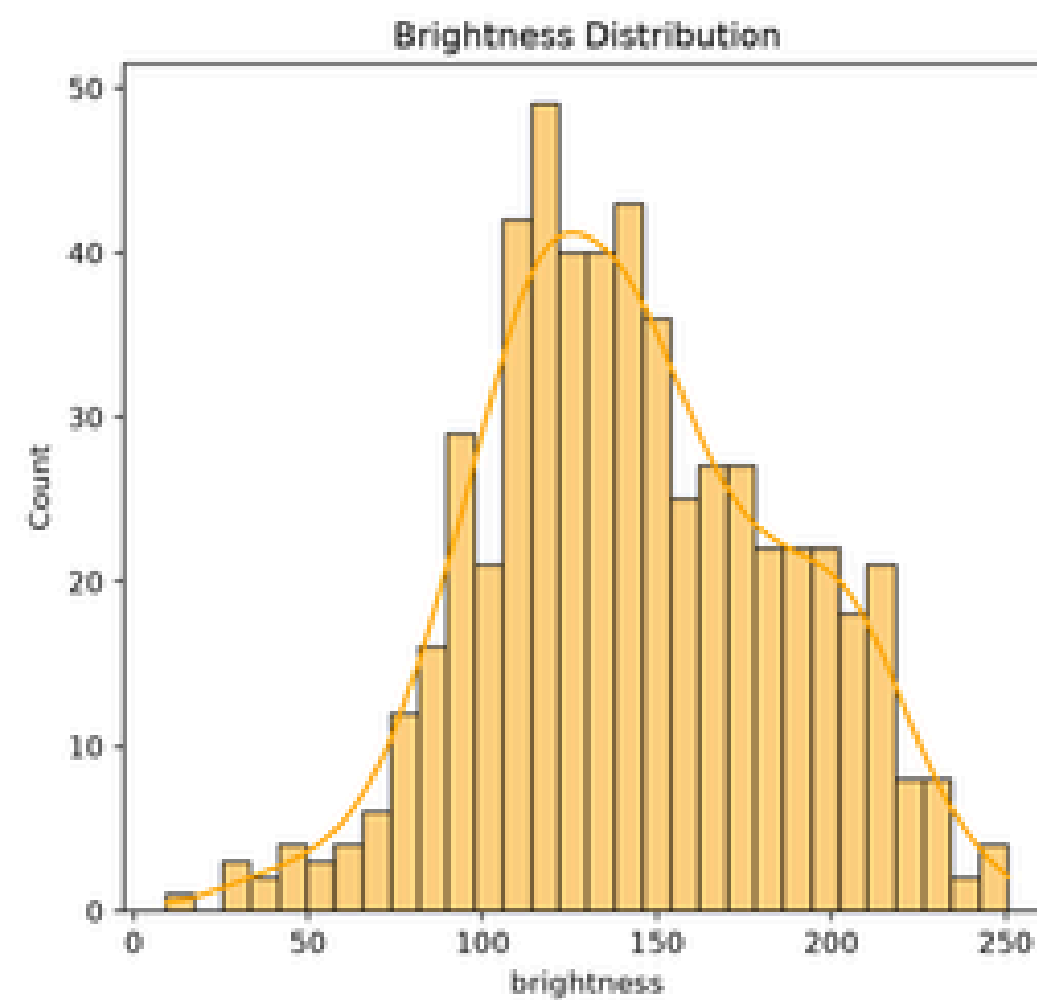
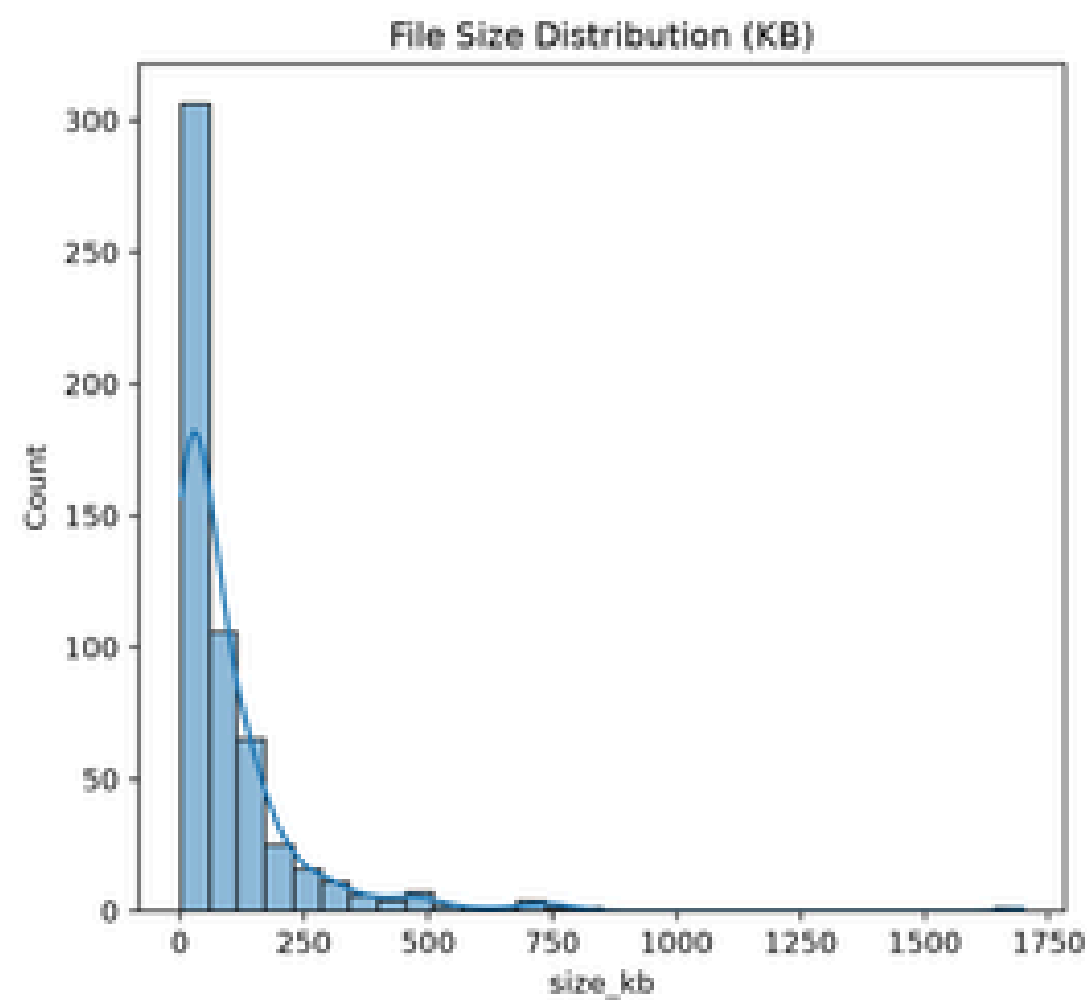
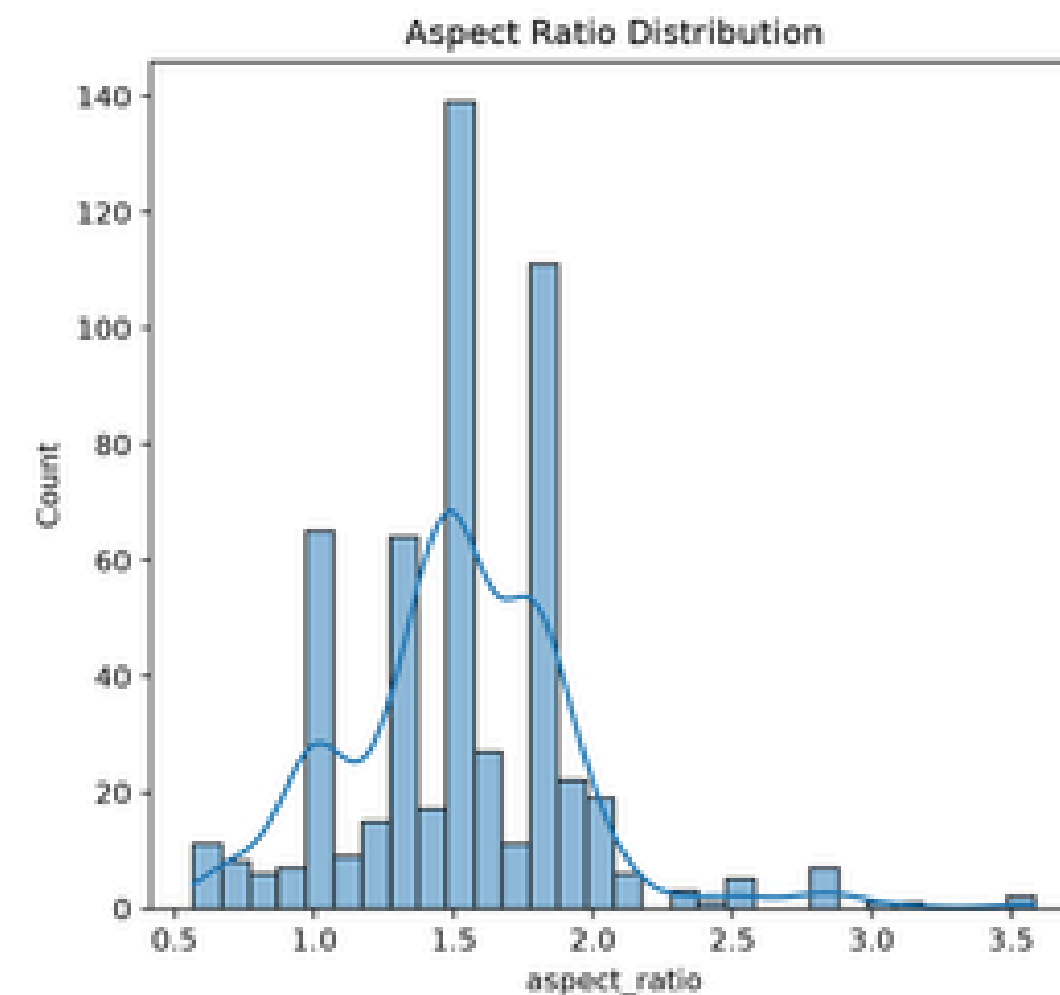
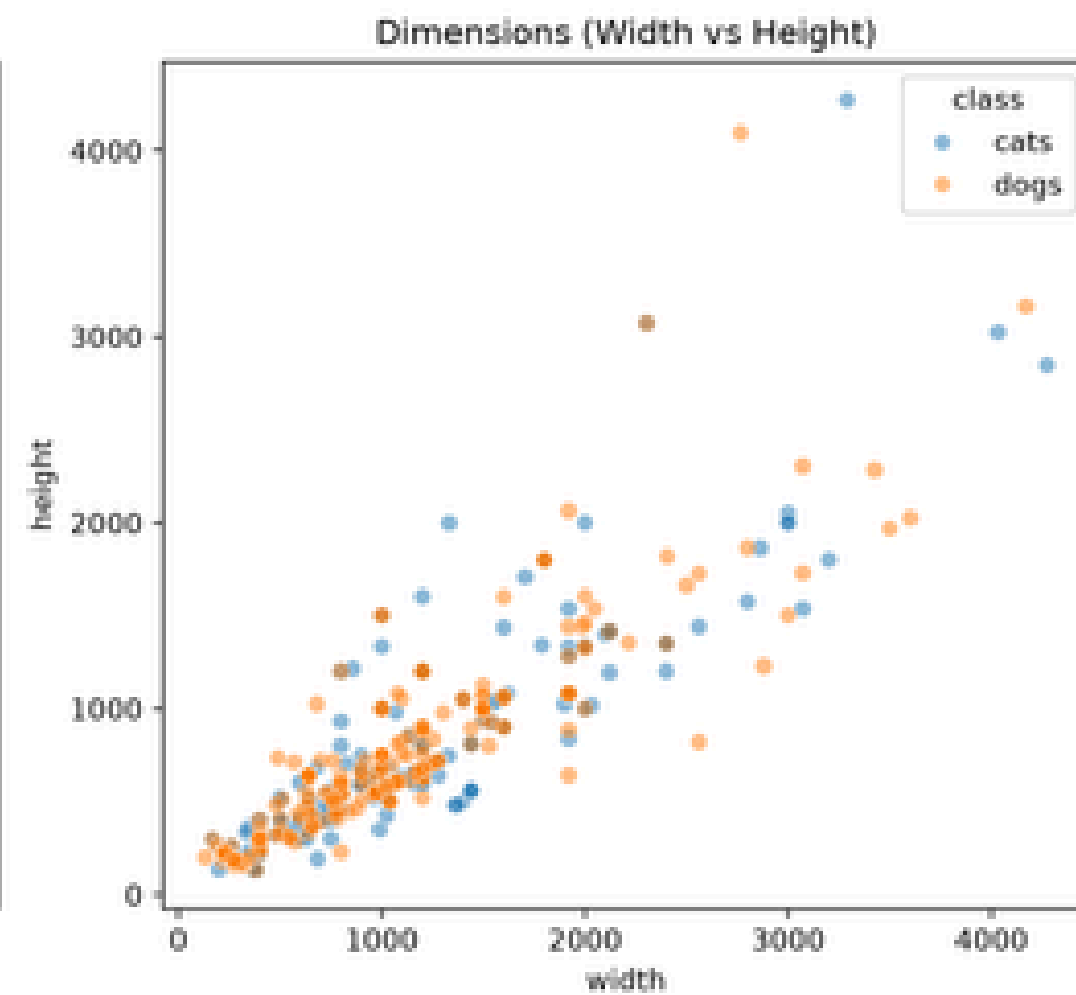
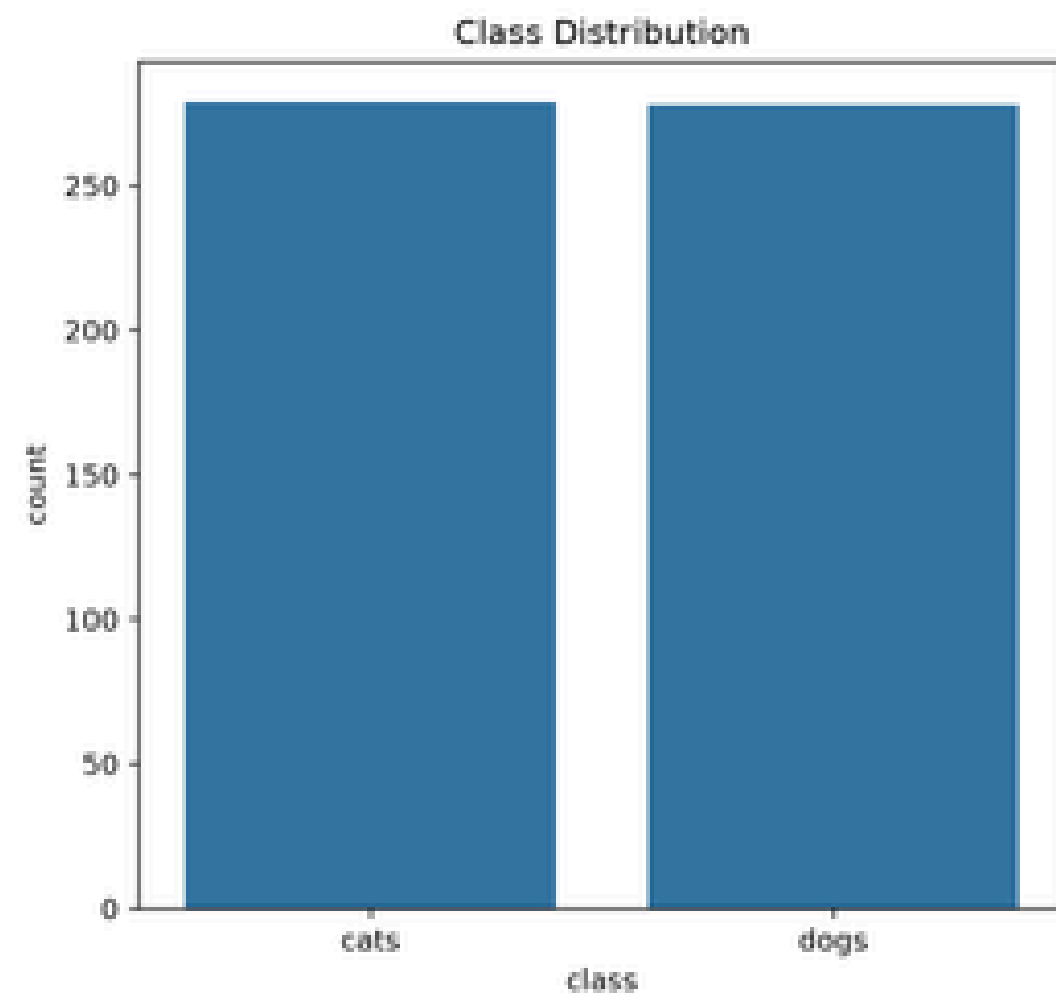
- Resize รูปภาพ
- ปรับ Aspect Ratio
- คัดภาพเสียออก
- คัดภาพซ้ำ
- จัดการภาพเบลอ
- จัดการภาพมืด/สว่างเกินไป

ในโค้ดของ EDA ถูกใช้เพื่อค้นหาเหล่านี้โดยตรง เช่น Class Imbalance, Aspect Ratio, Duplicate และ ภาพเบลอ/มืด/สว่าง ก่อนนำข้อมูลไป Train Model

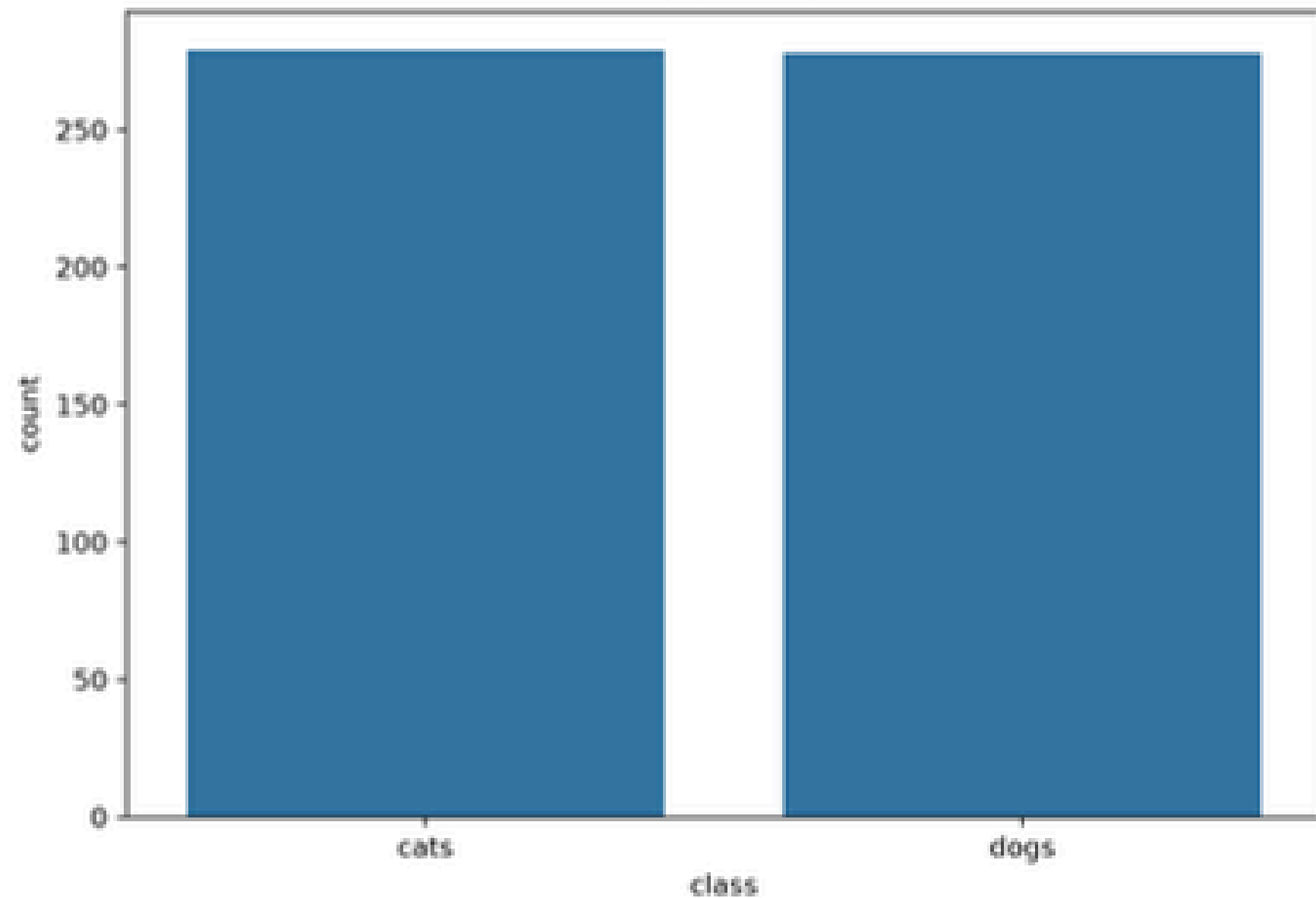


- วัตถุประสงค์ของการทำ EDA คือ เพื่อสำรวจและทำความเข้าใจ Dataset ก่อนนำไปสร้างโมเดล
- ตรวจสอบจำนวนข้อมูล คุณภาพของรูปภาพ ความสมดุลของแต่ละ Class
- ค้นหาข้อมูลที่ผิดปกติ เช่น รูปเสีย รูปซ้ำ รูปเบลอ หรือรูปที่มืดและสว่างเกินไป
- เราสามารถวางแผนการทำ Data Cleaning และ Preprocessing ได้อย่างเหมาะสม





Class Distribution



ความสมดุลของข้อมูล

มีจำนวนรูปภาพรวมทั้งหมด 555 ภาพ

แบ่งเป็นคลาสแมว Cats (278ภาพ) แบ่งเป็นคลาสหมา Dogs (277ภาพ)

สัดส่วนข้อมูลมีความสมดุลสมบูรณ์ (50:50)

Class Distribution

โค้ดสำหรับทำกราฟ

```
plt.subplot(2, 3, 1)
sns.countplot(data=df_valid, x="class")
plt.title("Class Distribution")
```

แบ่งพื้นที่ออกเป็นตาราง 2 แถว × 3 คอลัมน์
(รวมทั้งหมด 6 ช่องกราฟ)

1 ตัวสุดท้าย คือ การเลือกวาดกราฟลงใน "ช่องที่ 1"
(มุมซ้ายบนสุดของภาพแดชบอร์ด)

Dimensions (Width vs Height)

Scatter Plot แสดงความกว้าง (Width) คู่กับความสูง (Height) ของแต่ละรูป

แกน X – width สเกลเริ่มตั้งแต่ 0 ถึง 4,000 +
(กลุ่มจุดเริ่มตั้งแต่ประมาณ 200 จนไปถึง 4,200)

แกน Y – height สเกลเริ่มตั้งแต่ 0 ถึง 4,000+
(มีจุดพุ่งไปถึง 4,200)

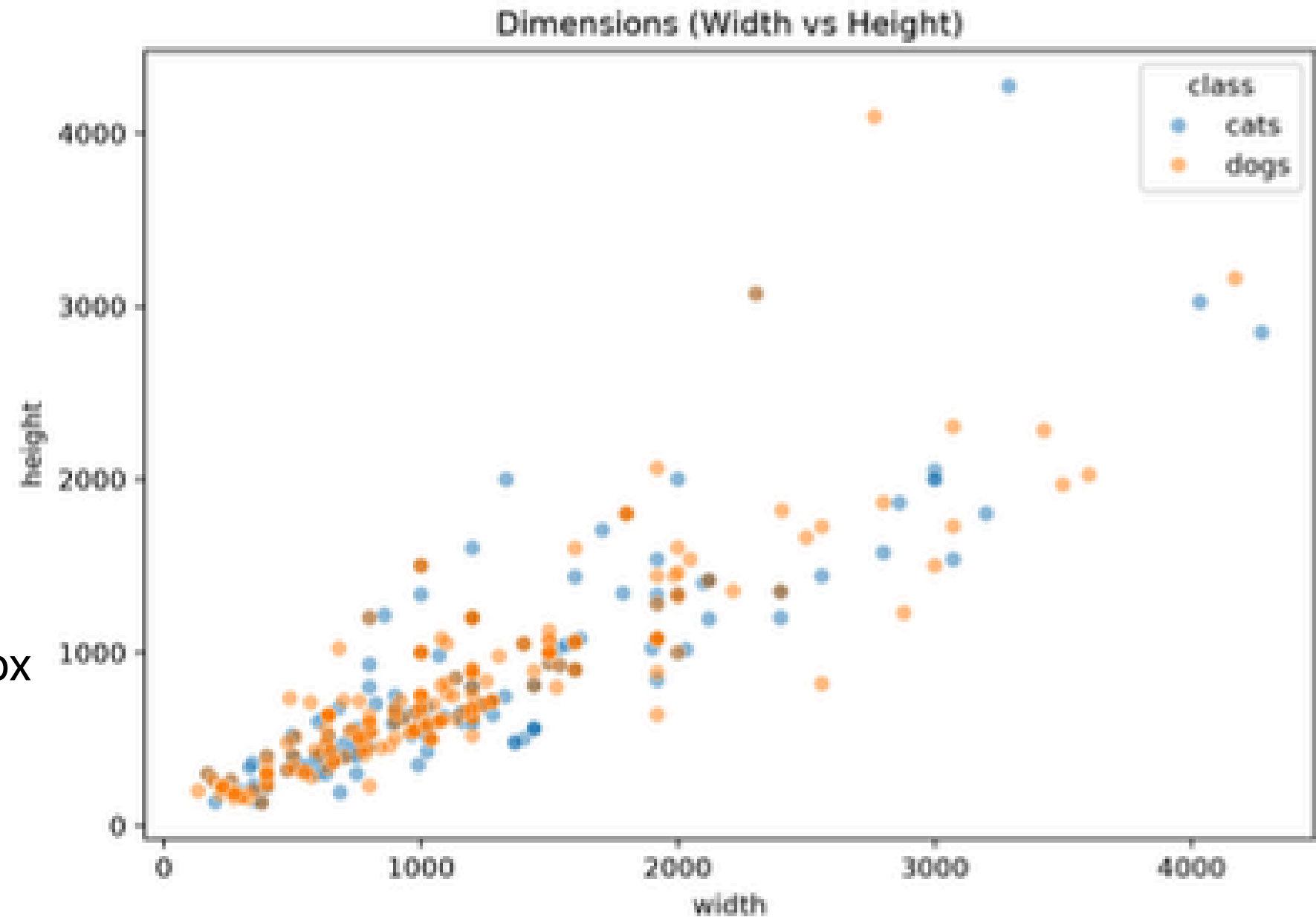
ภาพส่วนใหญ่กระจุกตัวอยู่ในช่วง 500x500 px ถึง 2000 x 2000 px

กลุ่มภาพส่วนใหญ่หนาแน่นมากที่สุดในช่วง

300x300 ถึง 1,200x1,200 px

มีบางภาพขนาดใหญ่มากพุ่งไปถึงระดับ

> 4000 px



Cats – สีส้ม
Dogs – สีน้ำเงิน

รูปที่มีความกว้างมาก มักจะมีความสูงมากตามไปด้วย

มีความสัมพันธ์ไปในทิศทางเดียวกันแบบเชิงเส้น (Positive Linear Correlation)

จนอาจทำให้โหลดเข้าโมเดลช้า หรือภาพที่เล็กเกินไปจนขาดรายละเอียดหรือไม่

Aspect Ratio

สัดส่วนความกว้างต่อความสูง (W/H)

Aspect Ratio คำนวณจาก $\text{Width} \div \text{Height}$

เช่น Width = 1500 Aspect Ratio = $1500 \div 1000$
Height = 1000 = 1.5

แกน X aspect_ratio ได้ทำคำนวณ

แกน Y จำนวนรูปภาพ `aspect_ratio = w / h`

แท่งกราฟแสดงว่ามีรูปจำนวน ที่มี Aspect Ratio ในช่วงนั้น

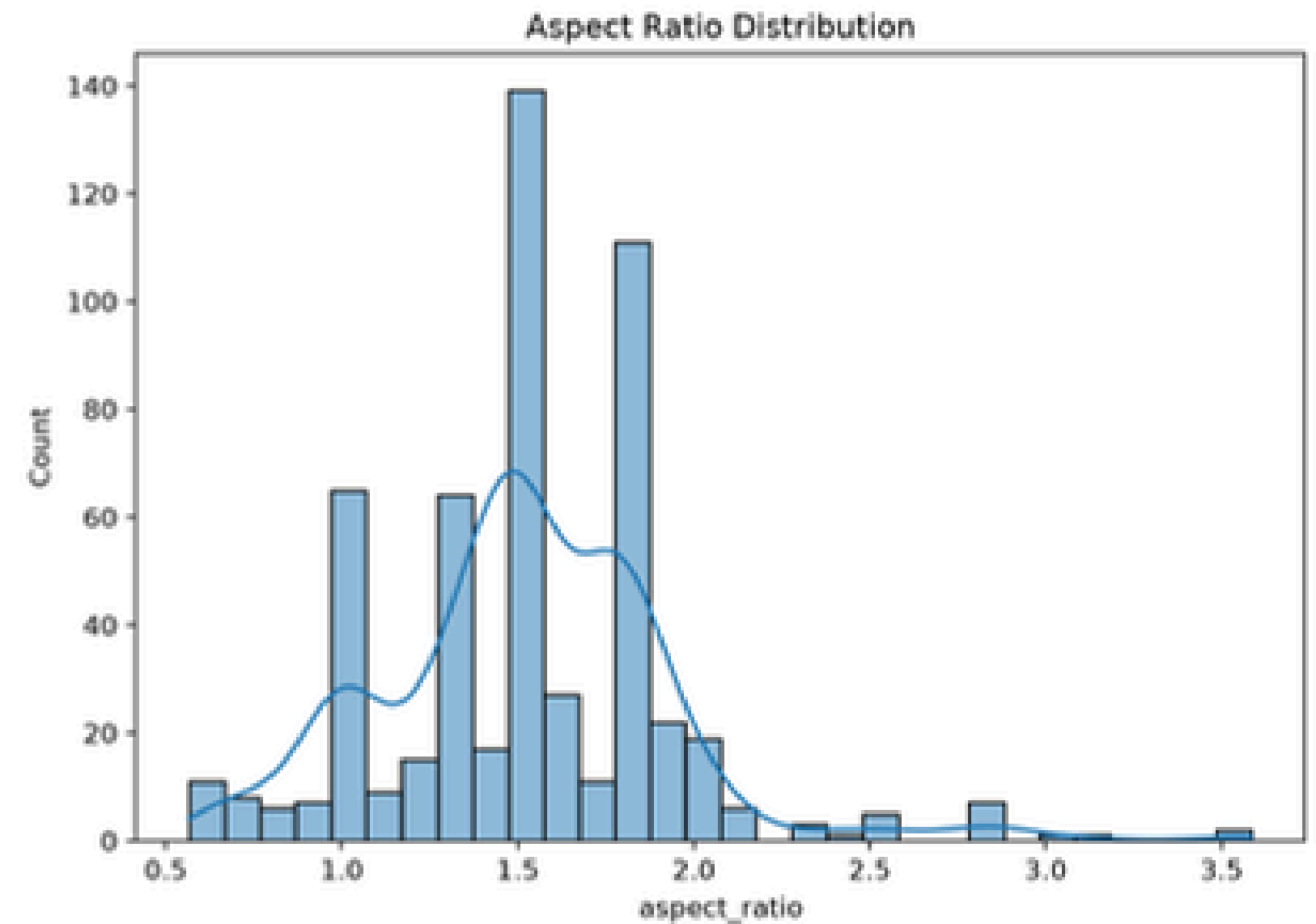
รูปส่วนใหญ่อยู่ประมาณ 1.0–2.0

แต่มีบางภาพที่มี Aspect Ratio สูงมาก เช่นประมาณ 2.5–3.5

ภาพเหล่านั้นมีลักษณะ กว้างเมื่อเทียบกับความสูง

เพราะถ้าเอาภาพที่มีสัดส่วนต่างกันมากไป Resize เป็นขนาดเดียวกันโดยตรง อาจทำให้ภาพ ยืดหรือบีบผิดรูป

ในโค้ดจึงเสนอว่าอาจใช้ Letterbox Padding หรือ Random Crop ก่อน Resize



File Size Distribution in KB

ไฟล์รูปภาพแต่ละรูปมีขนาดใหญ่แค่ไหน

แกน X = size_kb

แกน Y = จำนวนรูป

หน่วยของแกน X คือ KB

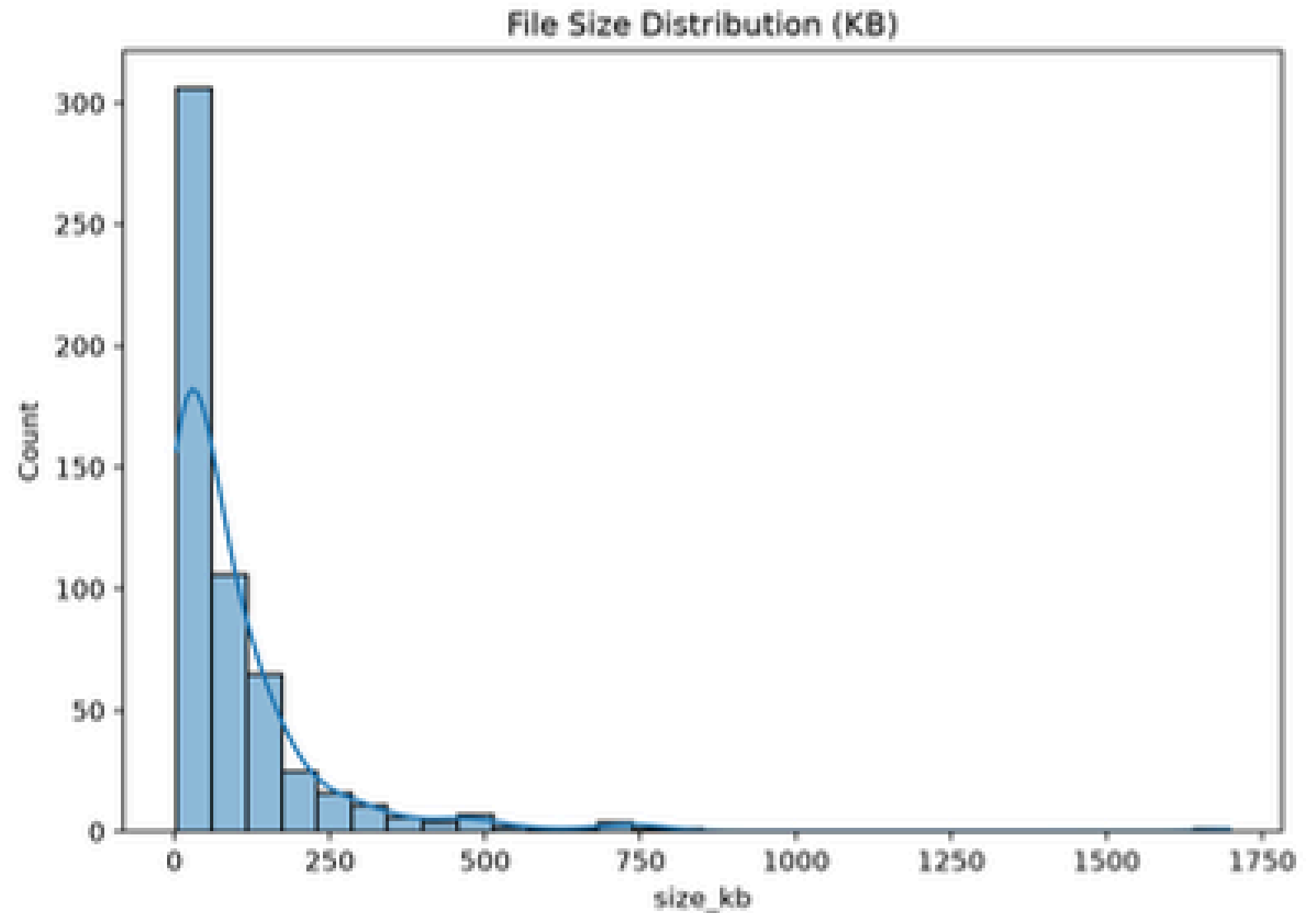
กราฟ

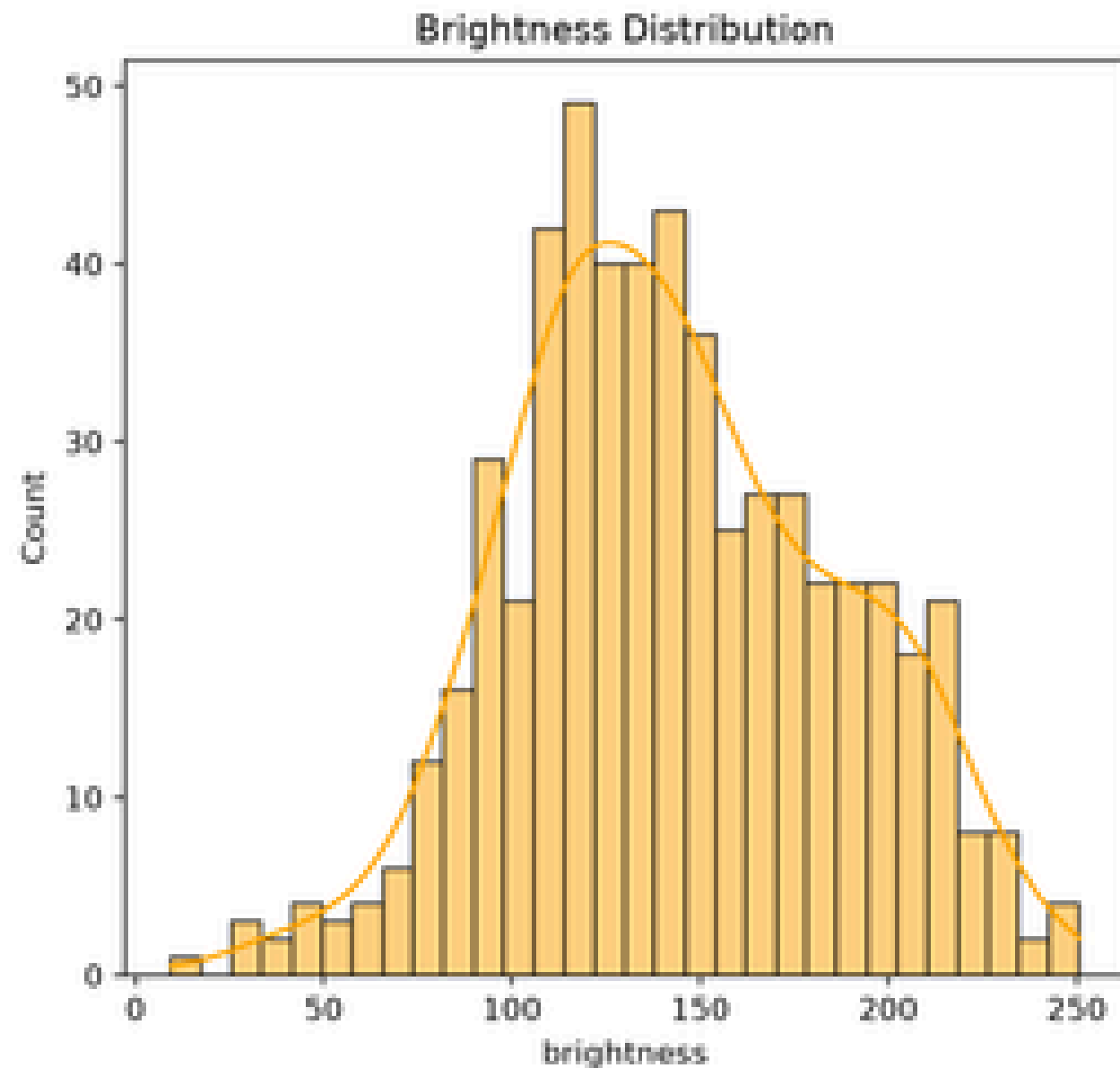
รูปส่วนใหญ่มีขนาดไฟล์ค่อนข้างเล็ก อยู่ทางด้านซ้ายของกราฟ แต่มีบางรูปที่มีขนาดไฟล์ใหญ่กว่ารูปอื่นมาจึงทำให้กราฟมีลักษณะยาวไปทางขวา

```
file_size_kb = os.path.getsize(img_path) / 1024
```

คือการหาขนาดไฟล์แล้วแปลงจาก Byte เป็น KB

เพราะถ้ามีไฟล์รูปขนาดใหญ่มากจำนวนมาก อาจทำให้ Dataset ใช้พื้นที่เยอะ โหลดรูปช้าขึ้น ใช้ RAM มากขึ้น Training ใช้เวลานานขึ้น





ข้อมูลส่วนใหญ่อยู่ประมาณช่วง 100–200

โดยมีจำนวนมากบริเวณประมาณ 120–150

รูปส่วนใหญ่มีความสว่างระดับกลางถึงค่อนข้างสว่าง

Brightness Distribution

ระดับความสว่างเฉลี่ยของภาพ

ภาพใน Dataset มีความสว่างมากหรือน้อย
ค่า Brightness ใช้ช่วงประมาณ 0–255

0 → มืด
255 → สว่าง

โค้ด

โค้ดแปลงภาพเป็น Grayscale

```
gray = cv2.cvtColor(  
    img,  
    cv2.COLOR_BGR2GRAY  
)
```

จากนั้นหาค่าเฉลี่ยของ Pixel

```
brightness = float(np.mean(gray))
```


Blur Score Distribution

ระดับความคมชัดและความเบลอ

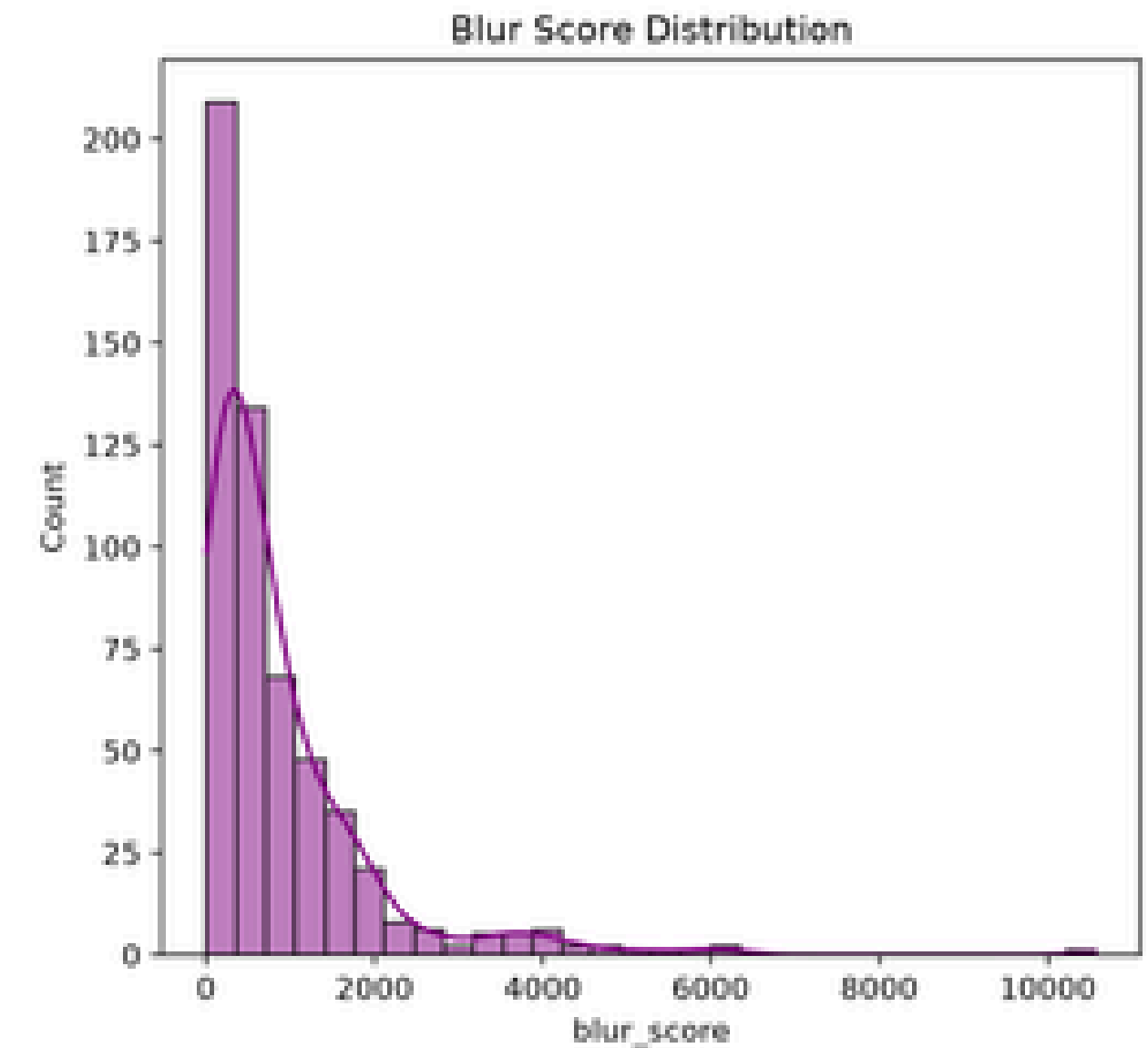
กราฟนี้ใช้ตรวจสอบว่า ภาพมีความเบลอมากน้อยแค่ไหน
ค่าที่ได้เรียกว่า Blur Score แต่จริง ๆ แล้วเป็นการวัด Variance ของ Laplacian

การวัด

ค่าต่ำ → ภาพมีแนวโน้มเบลอ

ค่าสูง → ภาพมีรายละเอียด/ขอบภาพมาก และมีแนวโน้มคมกว่า

เห็นว่ารูปจำนวนมากอยู่บริเวณค่า Blur Score ต่ำ
แล้วจำนวนรูปจะลดลงเรื่อย ๆ เมื่อค่า Blur Score สูงขึ้น
มีบางรูปที่มีค่ามากกว่า 2,000, 4,000 ไปจนถึง 10,000+
ดังนั้น Dataset มีภาพที่มีระดับความคมแตกต่างกันค่อนข้างมาก



โค้ด

```
blur_score = float(  
    cv2.Laplacian(gray, cv2.CV_64F).var()  
)
```

เพราะภาพที่เบลอมากอาจทำให้ Model ดึง Feature ที่สำคัญจากภาพได้ยาก
โค้ดจึงมีฟังก์ชันค้นหา Most Blurry เพื่อเอาภาพที่มีค่า Blur Score ต่ำที่สุดออกมาตรวจสอบด้วยสายตา

Qualitative Inspection

กราฟสำหรับตรวจสอบคุณภาพภาพแบบรายตัวอย่าง (Qualitative Inspection)
ต่างจากกราฟ 6 อันก่อน เพราะกราฟนี้ไม่ได้ดู Dataset ทั้งหมด

สุ่มรูปแมวมา 3 รูป แล้วดูภาพจริงคู่กับ Histogram ของสี
ได้ด เพื่อสุ่มตัวอย่างภาพแต่ละ Class

```
plot_and_save_samples(df, n_samples=3)
```

ด้านซ้าย

คือ ภาพจริง

ด้านขวา

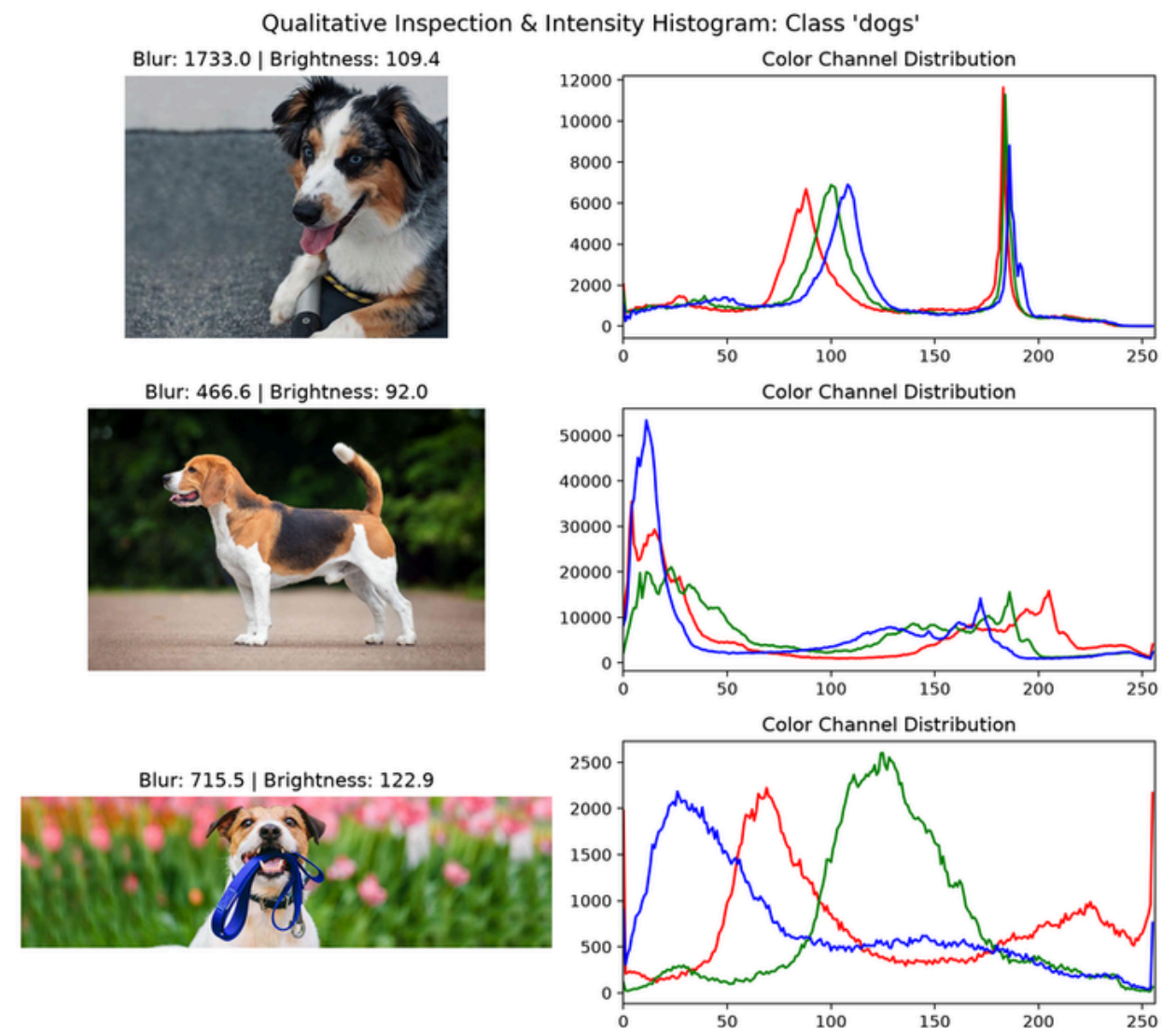
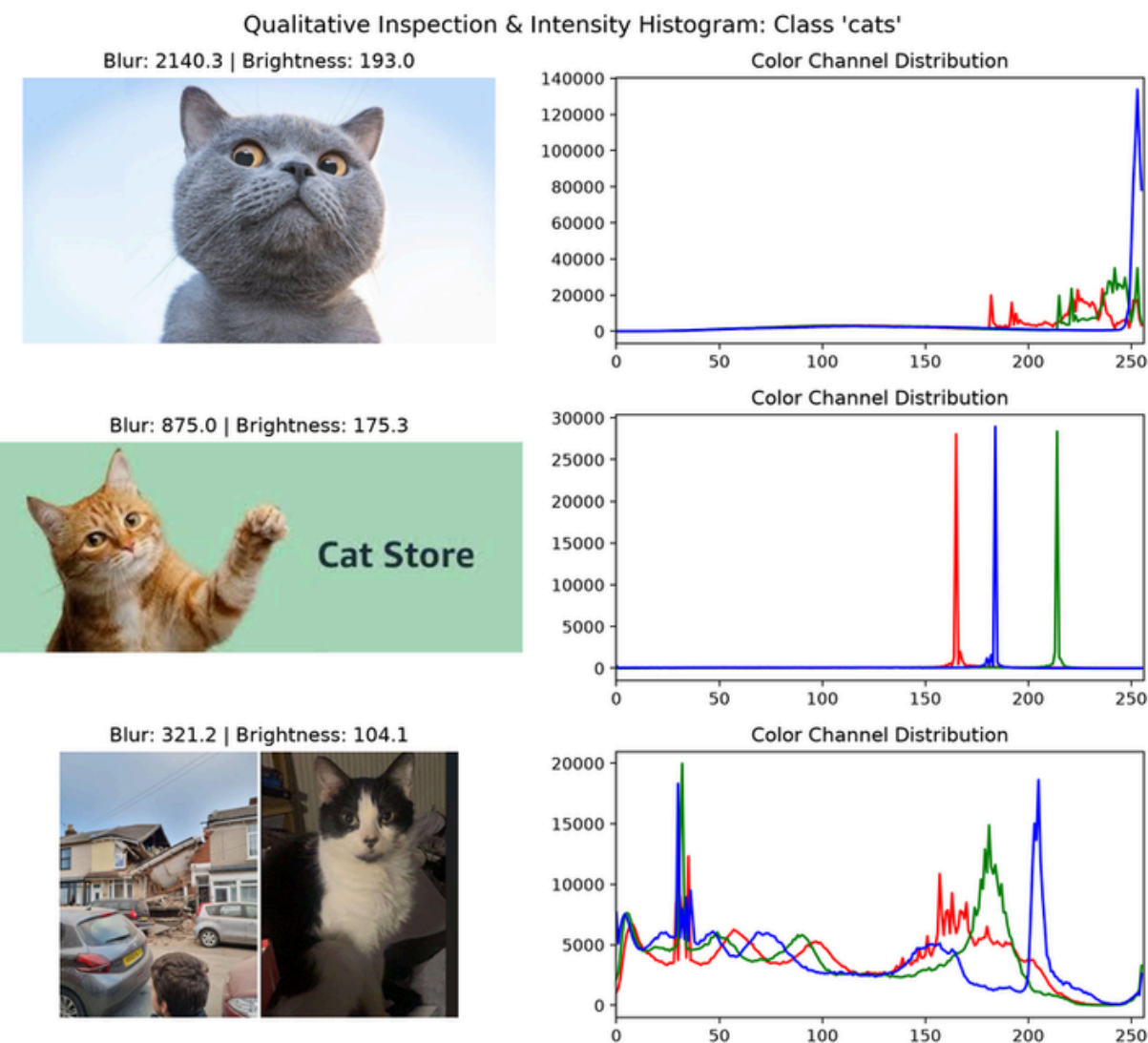
คือ Color Channel Distribution

ใช้ดูการกระจายของสี

- Red
- Green
- Blue

โดยแกน X = ค่าความเข้มของสี 0–255

แกน Y = จำนวน Pixel ที่มีค่านั้น



รูปที่ 1 แมว

ด้านบนสุดเขียนว่า

Blur: 2140.3 | Brightness: 193.0

Brightness = 193.0

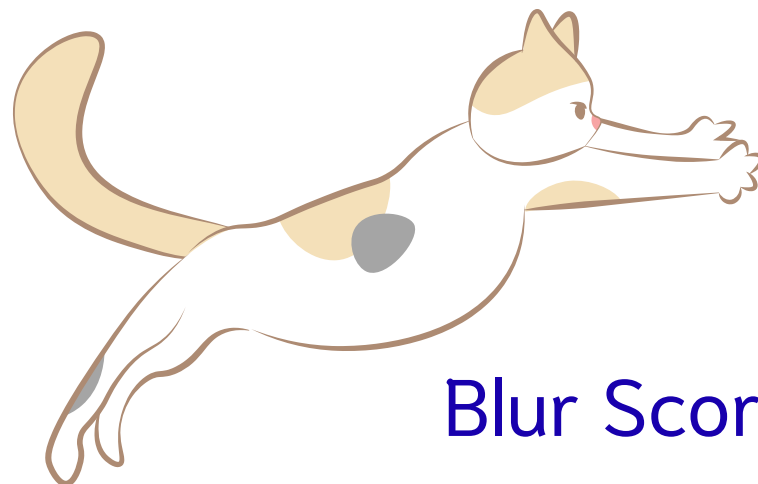
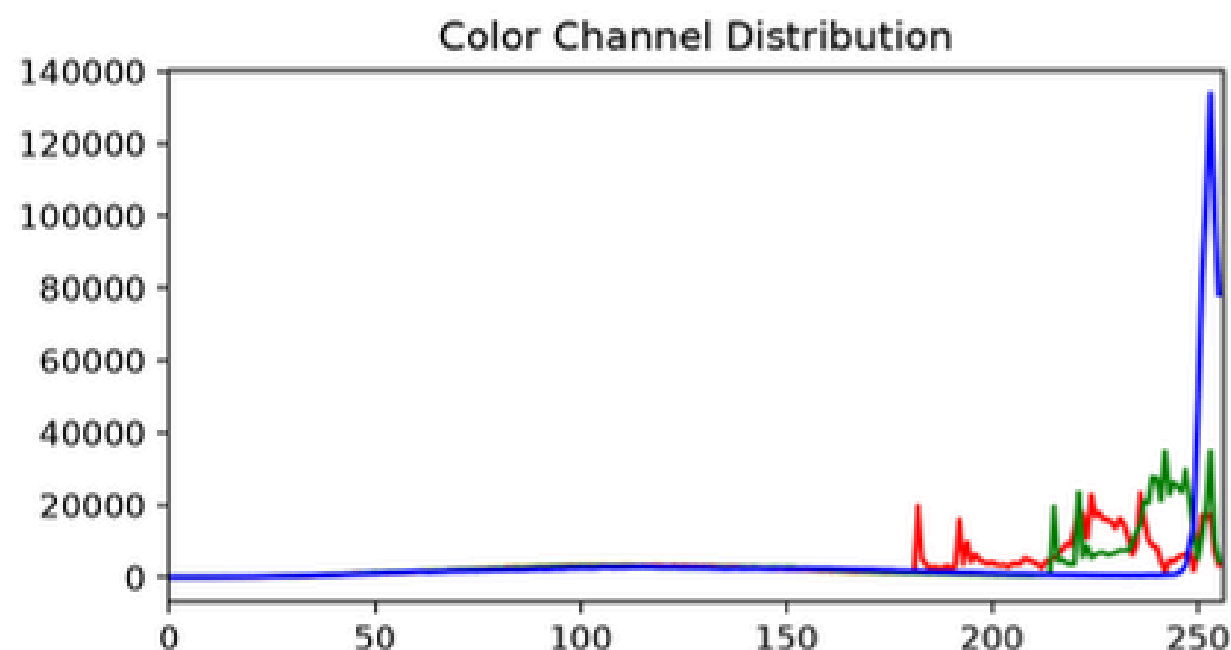
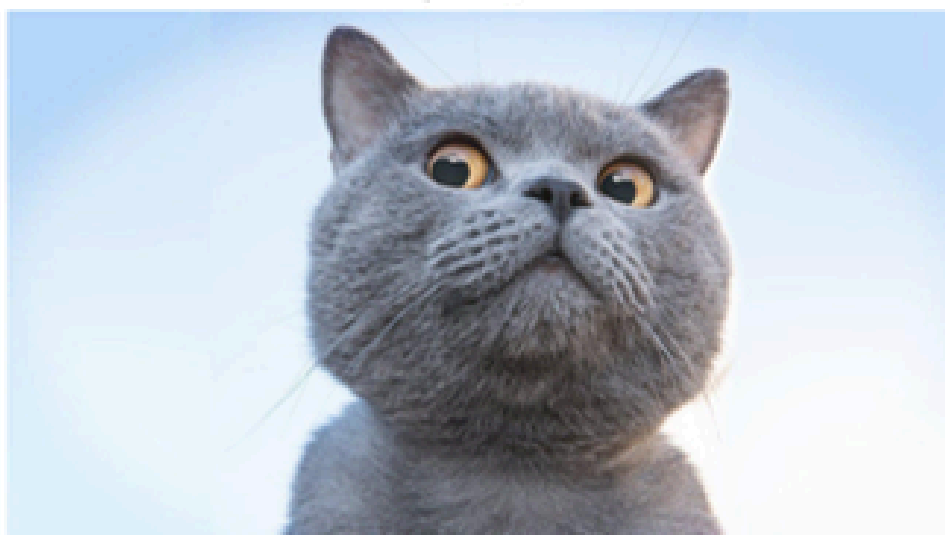
ค่าความสว่าง 193 ถือว่าค่อนข้างสูงเมื่อเทียบกับช่วง 0–255 จากภาพจะเห็นว่า

- พื้นหลังค่อนข้างสว่าง
- มีบริเวณสีขาว/ฟ้าอ่อนจำนวนมาก
- ตัวแมวมีสีเทาเข้มกว่า Background

ดังนั้นค่าเฉลี่ยความสว่างจึงค่อนข้างสูง

ภาพนี้มี Pixel ที่มีค่าความเข้มสูงจำนวนมาก โดยเฉพาะบริเวณสีฟ้า/สว่างของพื้นหลัง

Blur: 2140.3 | Brightness: 193.0



Blur Score = 2140.3

ค่า Blur Score ของรูปนี้ค่อนข้างสูงเมื่อเทียบกับตัวอย่างอีก 2 รูปตามวิธีที่ได้ใช้ คือ Laplacian Variance

ดังนั้นค่านี้สูงหมายถึงภาพมีขอบหรือรายละเอียดค่อนข้างมาก และมีแนวโน้มว่าภาพ คมกว่า ตัวอย่างที่มีค่าต่ำกว่า ดังนั้นใน 3 ตัวอย่างนี้



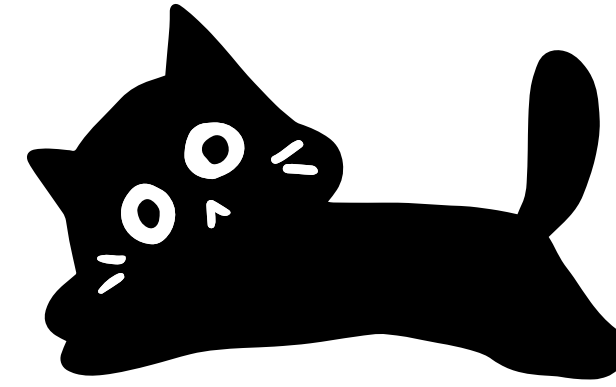
รูปที่ 1 หมา

ด้านบนสุดเขียนว่า

Blur: 1733.0 | Brightness: 109.4

Brightness = 109.4

ค่าอยู่ประมาณ 109 จากช่วง 0–255
ดังนั้นภาพนี้มีความสว่าง ระดับค่อนข้างมืดถึงปานกลาง
เมื่อดูจากภาพก็สอดคล้องกัน เพราะ Background เป็นพื้น
สีเทาเข้ม และบริเวณรอบ ๆ สุนัขไม่ได้สว่างมาก



Blur Score = 1733.0

ค่า Blur Score ค่อนข้างสูงเมื่อเทียบกับรูปสุนัขตัวอย่างที่ 2 และ 3
ดังนั้นตามวิธี Laplacian Variance ที่ใช้ในโค้ด ภาพนี้มีขอบและรายละเอียดค่อนข้างมาก จึงมีแนวโน้มว่า คมชัด

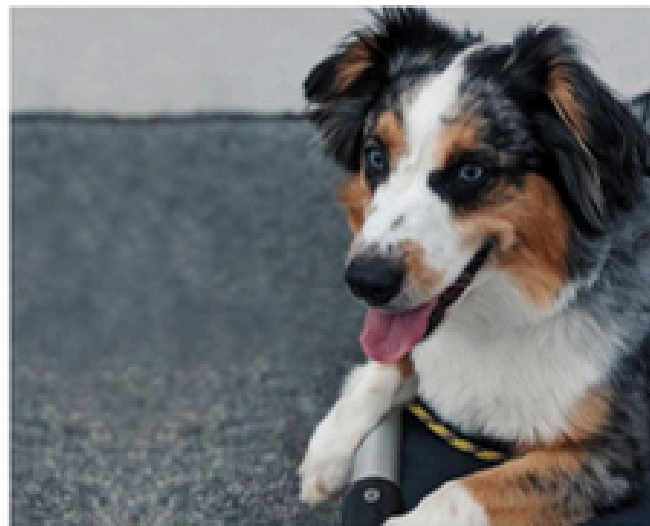
รูปที่ 1 มีแนวโน้มคมชัดที่สุดใน 3 ตัวอย่างของสุนัข

ภาพนี้มีความคมชัดค่อนข้างดี มีความสว่างระดับปานกลาง และมีการกระจายของสีที่สอดคล้องกับสีขาว เทา และสีเข้มของ
สุนัขกับพื้นหลัง

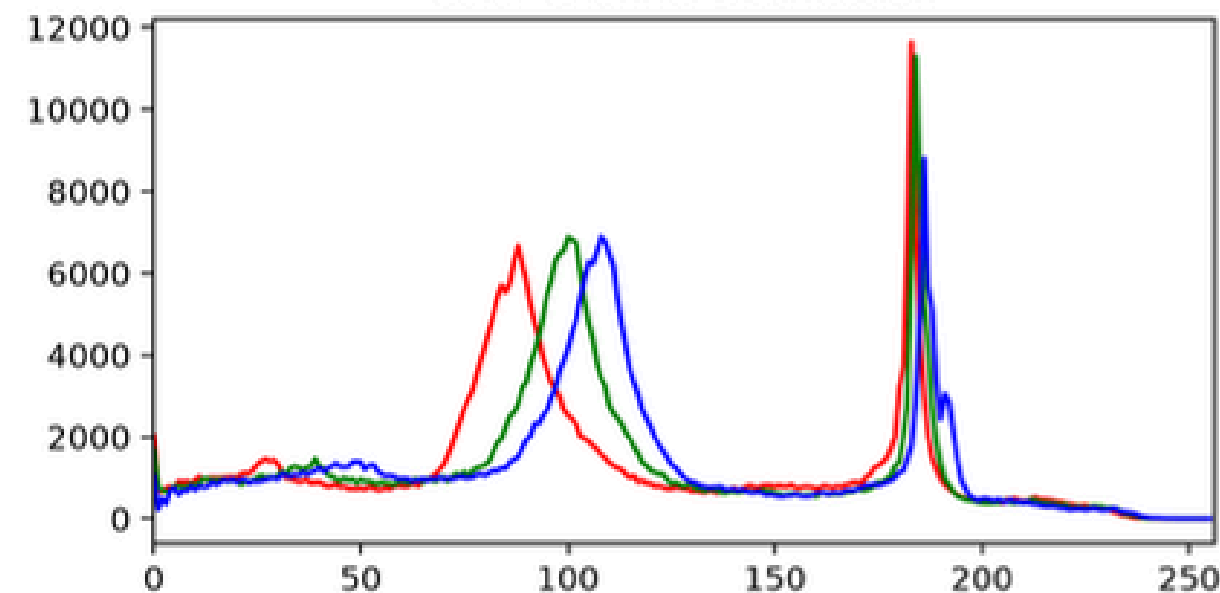


Qualitative Inspection & Intensity Histogram: Class 'dogs'

Blur: 1733.0 | Brightness: 109.4



Color Channel Distribution



คนที่ 3: Preprocessing + Image Processing (4.3, 4.4)

ไฟล์รับผิดชอบ: src/preprocessing.py, src/image_processing.py

ลบไฟล์เสีย, จัดการรูปซ้ำ (Hashing/Perceptual Hash)

จัดการ Class Imbalance (Oversampling/Undersampling/Weighted Sampling)

แปลง Format มาตรฐาน (.jpg/.png), แปลง Color Space (RGB)

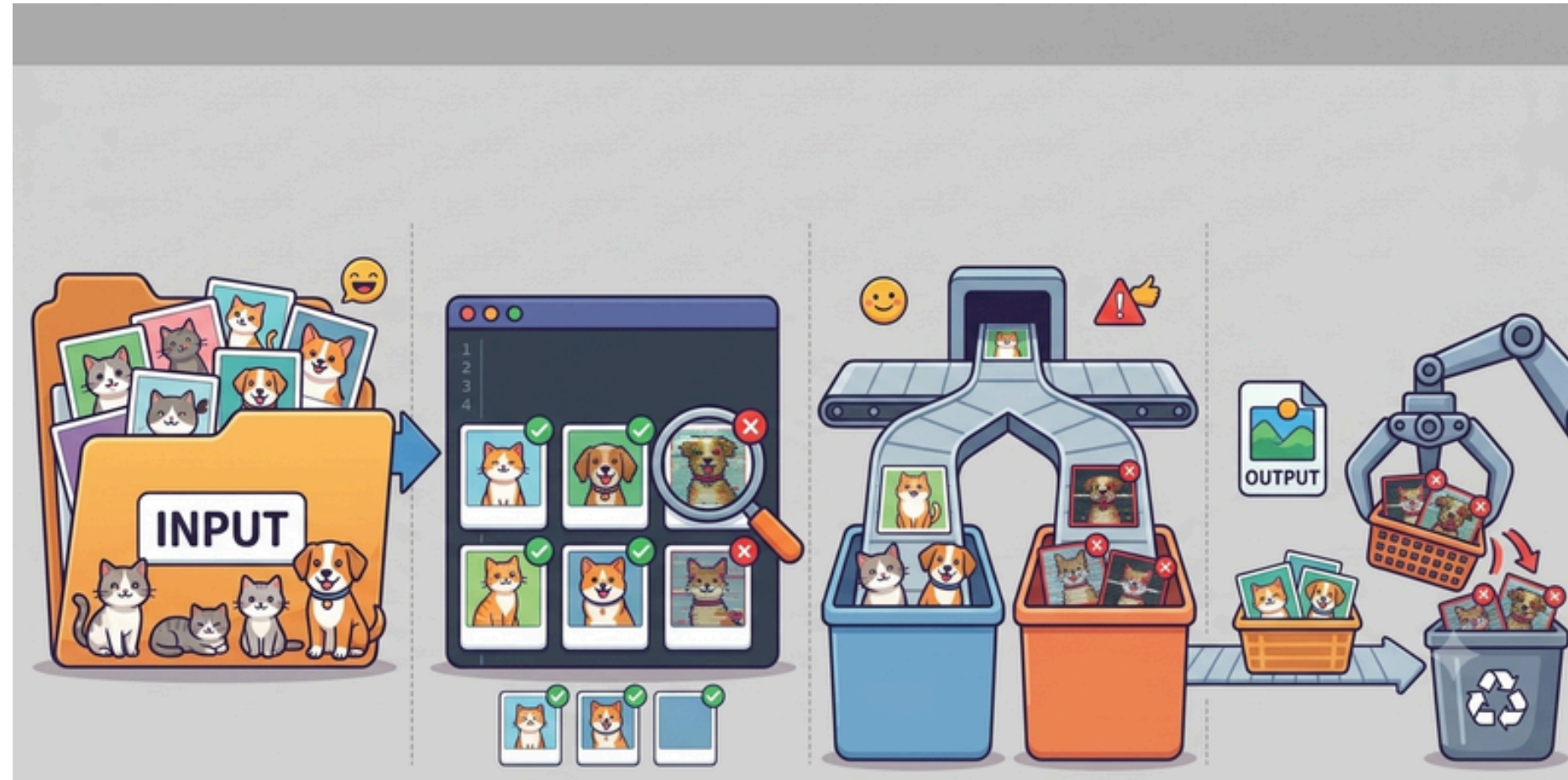
Resize ขนาดมาตรฐาน (พร้อมเหตุผล), Normalize/Standardize พิกเซล

Noise Reduction/Denoising ตามความเหมาะสม

Data Augmentation (flip, rotate, crop, brightness) พร้อมเหตุผล

แสดงภาพ Before/After ทุกขั้นตอน (เก็บใน reports/figures/)

Data Preprocessing



ลบไฟล์ที่เสียหายหรือใช้งานไม่ได้ (Corrupted Images)

โดยตรวจสอบความสมบูรณ์ของไฟล์ก่อนนำเข้าสู่
กระบวนการประมวลผล เพื่อป้องกันข้อผิดพลาดในขั้นตอน
ถัดไป

img.verify() เพื่อลองเปิดไฟล์หากเกิดErrorจะสั่ง
os.remove() ลบทิ้งทันที

--- สรุปผลการทำงาน ---

จำนวนไฟล์ภาพทั้งหมดที่ตรวจพบ: 697 ไฟล์

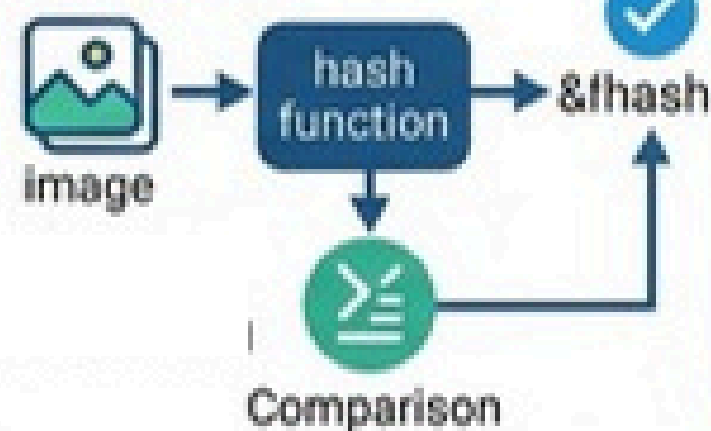
จำนวนไฟล์ที่เสียหายและถูกลบออก: 0 ไฟล์

Data Preprocessing

ข้อมูลภาพหมาและแมว (ต้นฉบับ)



✕ ตรวจสอบรูปภาพซ้ำ



ข้อมูลภาพหมาและแมว (หลังการประมวลผล)



ผลลัพธ์: Dataset ที่มีความหลากหลายสูงและไม่มีภาพซ้ำ

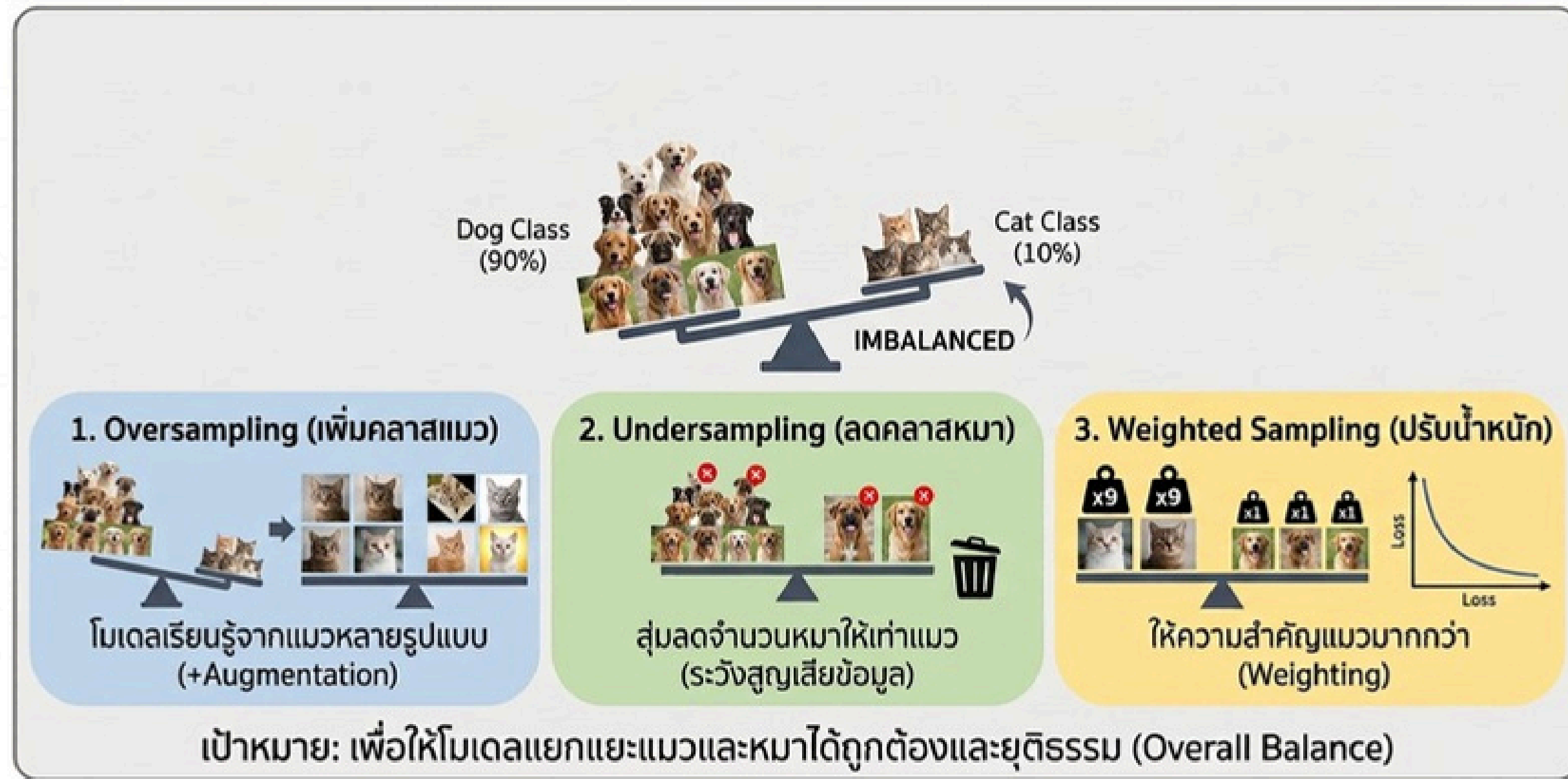
ตรวจจับและจัดการรูปภาพซ้ำ

โดยใช้เทคนิค Hashing หรือ Perceptual Hash เพื่อระบุและลบรูปภาพที่ซ้ำกันออกจาก Dataset

```
[X] รวบรวม: dog_73.jpg (link dog_33.jpg)
[X] รวบรวม: dog_525.jpg (link dog_305.jpg)
[X] รวบรวม: dog_596.jpg (link dog_266.jpg)
[X] รวบรวม: dog_77.jpg (link dog_514.jpg)
```

--- สรุปผลการตรวจจับภาพซ้ำ ---
จำนวนภาพทั้งหมดที่ตรวจพบ: 557 ไฟล์
จำนวนภาพซ้ำที่ถูกลบออก: 21 ไฟล์

Data Preprocessing



จัดการ Class ที่ไม่สมดุล (Class Imbalance)

เลือกใช้ Oversampling (ผ่าน Weighted Sampling) คู่กับ Data Augmentation เพื่อให้โมเดลเรียนรู้คลาสส่วนน้อยได้อย่างทั่วถึงโดยไม่เสียข้อมูลคลาสส่วนใหญ่

```
Pipeline Engine Ready!  
Total Images: 557  
Classes Found: ['cats', 'dogs']
```

```
Batch Tensor Shape : torch.Size([32, 3, 224, 224])  
Batch Labels Shape : torch.Size([32])  
Class Distribution : [20, 12]
```


Data Preprocessing

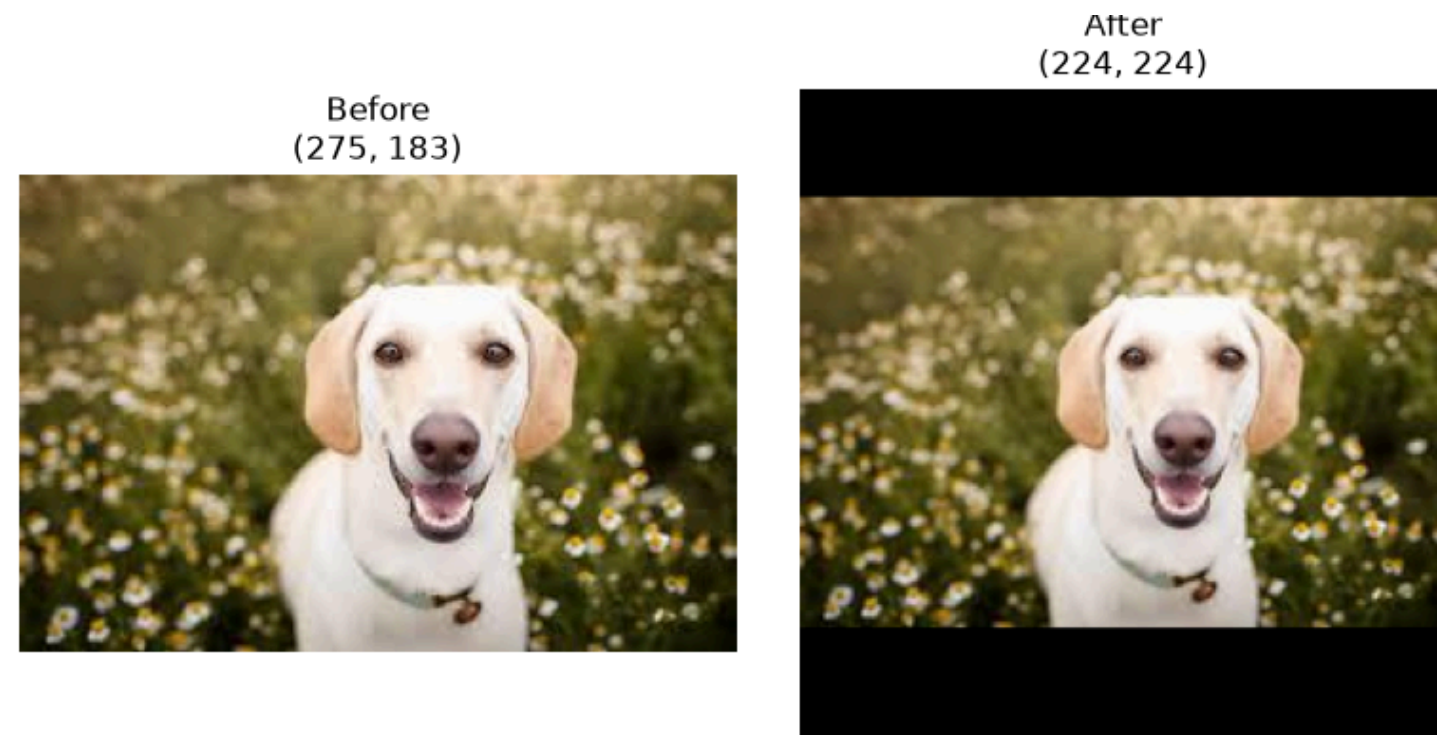


แปลงFormat ให้เป็นมาตรฐานเดียวกัน

dataset ดิบมีทั้ง .jpg/.png ปนกัน และมี mode สีไม่เหมือนกัน (Grayscale, RGBA, CMYK) ถ้าไม่ทำให้เป็นมาตรฐานเดียวกัน โมเดลจะรับ input ไม่ตรงมิติ/รูปแบบกัน อาจ error หรือเรียนรู้ผิดเพี้ยน

```
--- สรุปผลการแปลง Format & Color Space ---  
จำนวนไฟล์ที่แปลงเรียบร้อยแล้ว: 0 ไฟล์  
จำนวนไฟล์ที่เป็นมาตรฐานอยู่แล้ว: 536 ไฟล์  
จำนวนไฟล์ที่เกิดข้อผิดพลาด: 0 ไฟล์
```


Image Processing



Resize

ขนาดมาตรฐานที่นิยมใช้: 224 x 224 pixels

224×224 เป็นขนาดมาตรฐานที่โมเดล

pretrained ยอดนิยม (ResNet, VGG, MobileNet)

ถูกเทรนมา ถ้าจะทำ Transfer Learnin ต้องขนาดเท่ากัน

Image Processing

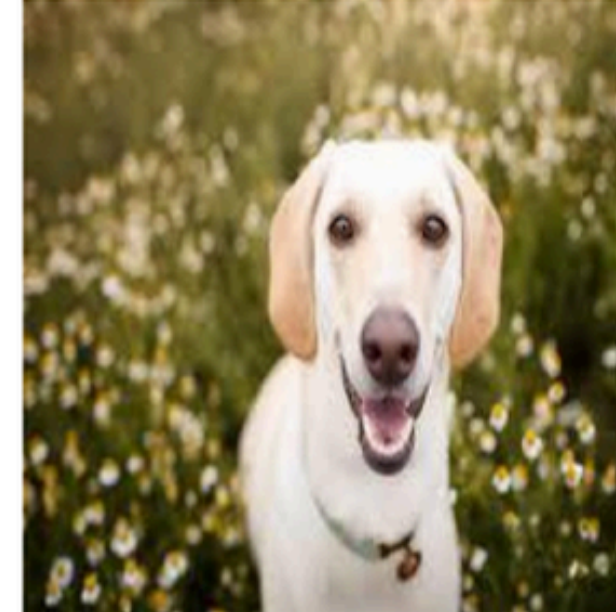
BEFORE: Raw Image (With Noise)



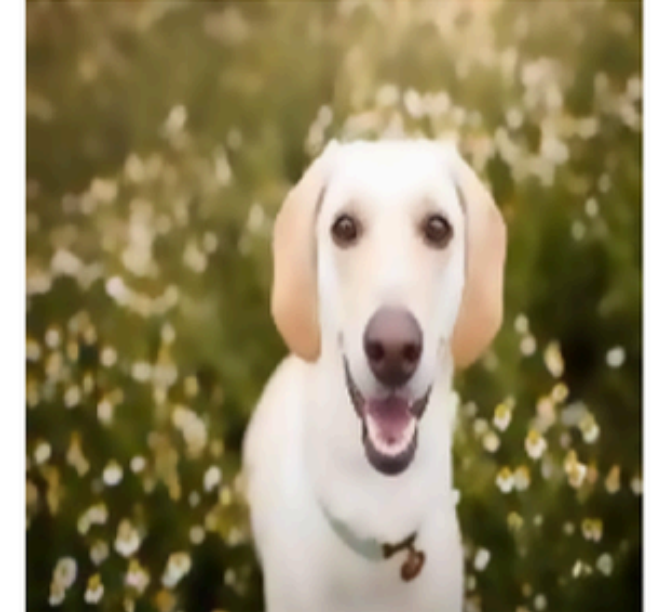
AFTER: Denoised Image (Bilateral Filter)



BEFORE: Raw Image (With Noise)



AFTER: Denoised Image (Bilateral Filter)



Denoising (Median Filtering)

ภาพที่มีจุด Noise / รอยรบกวน

ลด Noise ในรูปภาพด้วยเทคนิค Denoising ที่
เหมาะสมกับ Dataset Gaussian Blur เพื่อเพิ่ม
คุณภาพของข้อมูลก่อนนำเข้า Model

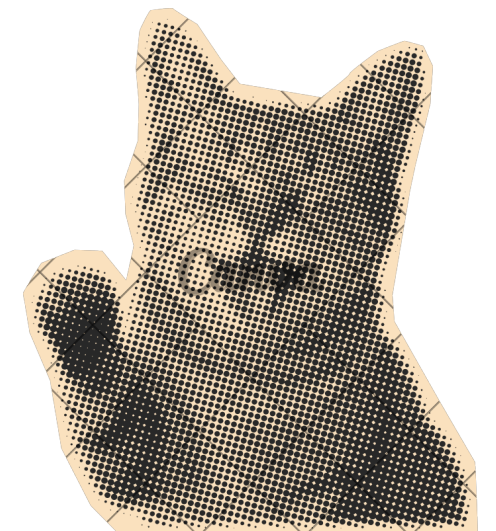
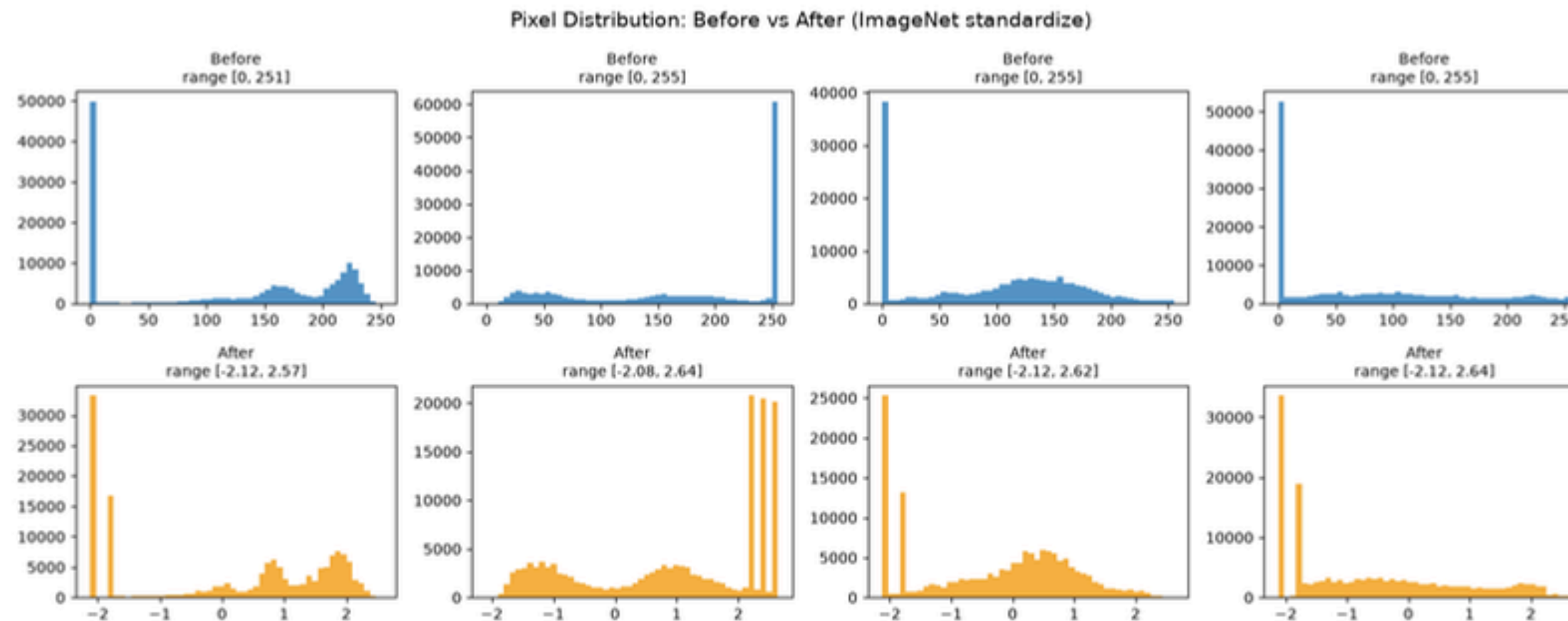


Image Processing



Normalization / Standardization ค่าพิกเซล

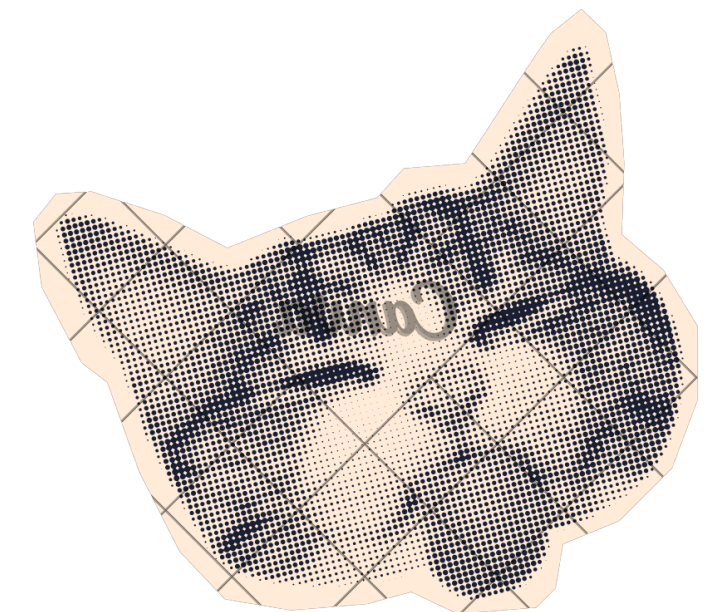
ปรับค่าพิกเซลให้อยู่ในช่วงมาตรฐาน เช่น
Scale เป็น 0–1 หรือใช้ Mean–Std
Normalization เพื่อให้ Model เรียนรู้ได้เร็วขึ้น
และมีเสถียรภาพในการ Train

Image Processing

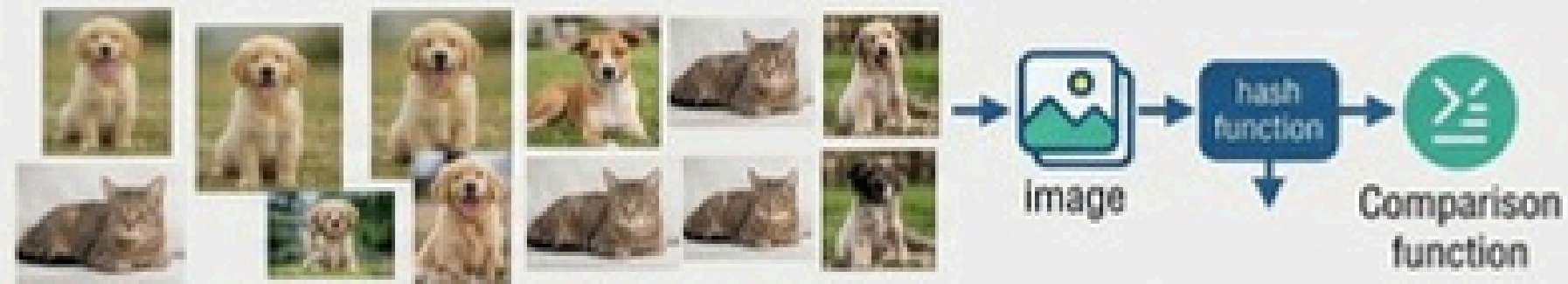


Data Augmentation (Flip & Brightness)

เพิ่มความหลากหลายของข้อมูลด้วย Flip, Rotate, Crop, Brightness Adjust เพื่อป้องกัน Overfitting และเพิ่มความสามารถในการ Generalize ของ Model



PREVIOUSLY: Preprocessing (Hashing)

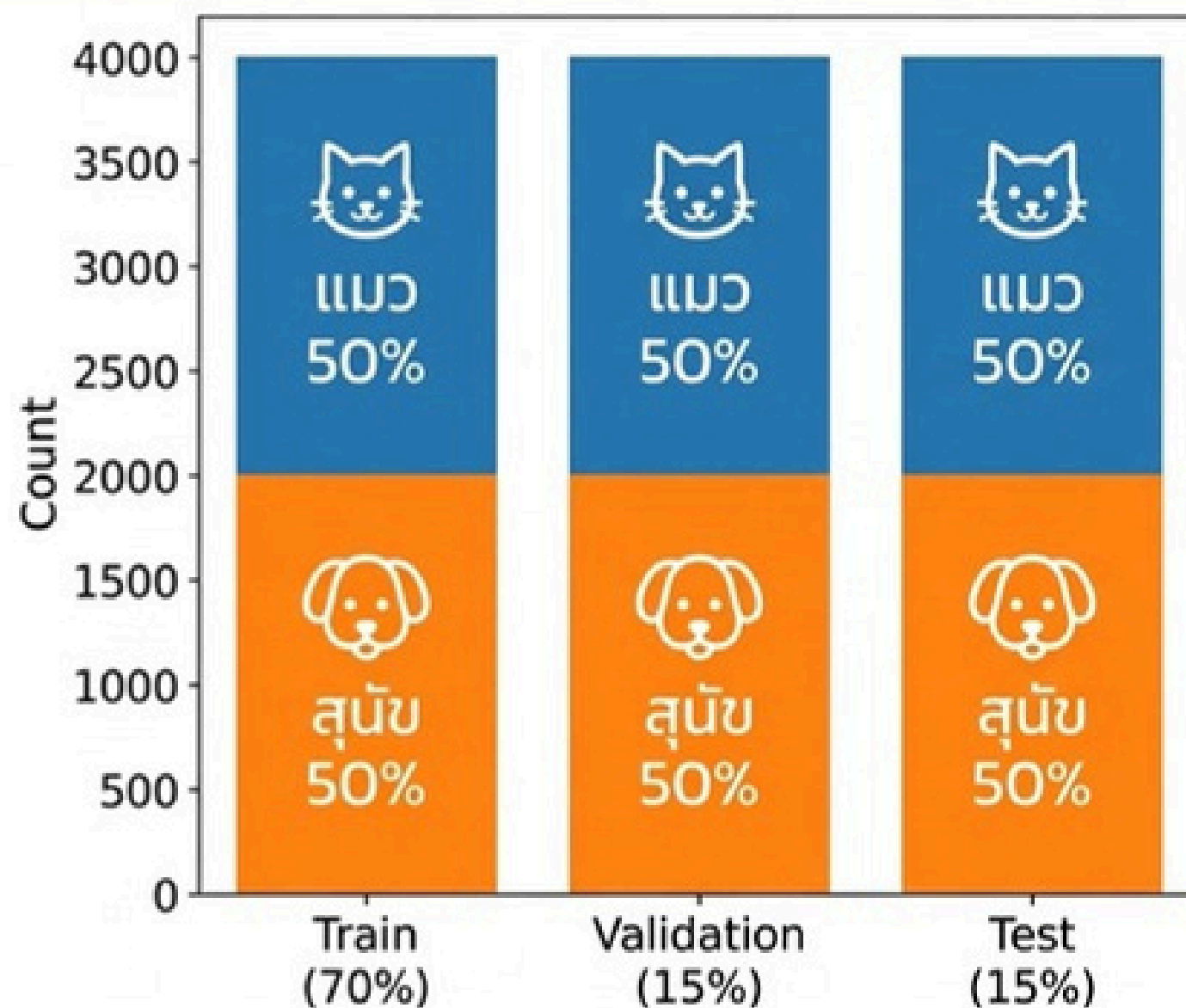


Data Splitting & Usage



แบ่งข้อมูลรูปภาพทั้งหมดออกเป็น 3 ส่วน ในสัดส่วน 80 : 10 : 10 โดยใช้เทคนิค Stratified Split

7.6 กราฟยืนยันสัดส่วน Class สมดุล (Stratified Split)



✓ สัดส่วนเท่ากันทุกชุด 

ทั้ง Train, Val, Test มี
แมว/สุนัข อย่างละ 50%

✓ ไม่มี Class ไหนเยอะกว่า

แบ่งข้อมูลแบบ Stratified
เพื่อรักษาความสมดุล 

✓ โมเดลเรียนรู้อย่างยุติธรรม

ประเมินผลได้แม่นยำ
ไม่เอียงไปทางคลาสใดคลาสหนึ่ง

ถ้าข้อมูลไม่สมดุล จะเกิดมีโอกาสผิดพลาด

AI จะเกิดความลำเอียง (Bias): ถ้าตอหนฟีก AI เห็นรูปสุนัขบ่อยกว่า มันจะจำพฤติกรรมว่า "ถ้าไม่แน่ใจ ให้ทายว่าสุนัขไว้ก่อน" ทำให้การเดาขาดความแม่นยำ และทำให้มีผลลัพธ์ผิดได้

ความพร้อมสำหรับประเภทงาน

- พร้อมสำหรับ Image Classification (100%): เหมาะสำหรับงานจำแนกประเภทรูปภาพ (ระบุว่าเป็นภาพ "แมว" หรือ "สุนัข") เนื่องจากข้อมูลมี Label กำกับไว้ในระดับตัวภาพเรียบร้อยแล้ว
- ยังไม่พร้อมสำหรับ Object Detection หรือ Segmentation: เนื่องจากไม่มี Bounding Box (กรอบระบุตำแหน่งวัตถุ) หรือ Mask หากต้องการนำไปใช้ทำ Detection จำเป็นต้องนำภาพไปทำ Annotation (ติ๊กเส้นระบุตำแหน่ง) เพิ่มเติมเองก่อน

ข้อจำกัดของชุดข้อมูล

- ความหลากหลายของสภาพแวดล้อมมีจำกัด: ส่วนใหญ่เป็นภาพถ่ายสัตว์เลี้ยงในบ้านหรือสตูดิโอ หากนำไปประมวลผลภาพในชีวิตจริง เช่น สัตว์จรจัด ภาพย้อนแสง ภาพเบลอ หรือมีวัตถุบังตัวสัตว์ ความแม่นยำอาจลดลง



สรุปและต่อยอด

ปัญหาที่พบบ่อยระหว่างทำงาน

- พบไฟล์ภาพที่ไม่สมบูรณ์, ขนาดภาพไม่เท่ากัน
- ปัญหา Overfitting ในช่วงแรกเนื่องจากมุมมองและแสงของภาพสัตว์เลี้ยงที่มีความหลากหลาย

ต่อยอดในงานถัดไป

- Deploy เป็น Web/App Demo: นำโมเดลไปพัฒนาต่อเป็นเว็บแอปพลิเคชัน
- ต่อยอดไปสู่การระบุตำแหน่ง (Bounding Box) ของแมวและสุนัขในภาพด้วย YOLO หรือ SSD

สิ่งที่ต้องการปรับปรุงหากมีเวลา

- เพิ่มปริมาณข้อมูล (Data Augmentation) เพื่อครอบคลุมแสงและมุมมองที่หลากหลายยิ่งขึ้น
- ปรับแต่ง Hyperparameters (เช่น Learning Rate, Batch Size และ Optimizer)





จบการนำเสนอ

THANK YOU

